# A Fifth-Order POE-Based Method for Kinematic Identification and Inverse Kinematics of Serial Robots

Yuhan Chen, *Member, IEEE*, Yunkai Wang, Fan Bu, Guiyang Xin, *Member, IEEE*, Changsheng Dai, *Senior Member, IEEE*, Xingjian Liu, *Member, IEEE*, Yu Sun, *Fellow, IEEE*, and Xinyu Liu, *Member, IEEE*

***Abstract*—Current numerical methods for solving kinematic identification (KI) and inverse kinematics (IK) are limited in accuracy, convergence rates, and robustness, necessitating further enhancement. This paper presents a modified Halley method for solving the KI and IK problems of serial robots based on the product of exponentials formula, achieving quintic convergence. Specifically, a general error model is first established based on exponential coordinates, and KI and IK are reformulated as root-finding problems. Next, the modified Halley method, which we prove to be a fifth-order method and incorporates a damping strategy, is proposed to resolve the singularity issue and enhance robustness. Subsequently, the Jacobian and Hessian matrices required for the proposed method are analytically derived based on the time differential of exponentials. Furthermore, highly simplified explicit formulas for these matrices are presented for the IK problem. Simulations on serial robots with various configurations validate the proposed method's accuracy, convergence rates, and robustness in solving KI and IK problems, as well as its advantages over the state-of-the-art. Additionally, experimental validation of KI on two physical robots further demonstrates the effectiveness of the proposed method. Our custom-written MATLAB and C++ codebases are made publicly available for download.**

***Index Terms*—Kinematic identification and calibration, inverse kinematics, product of exponentials (POE), Halley's Method, Hessian matrix.**

## I. Introduction

BOTH kinematic identification (KI) and inverse kinematics (IK) can be classified as robotic kinematic problems. The former aims to compensate for the robot's manufacturing and assembly tolerance, while the latter determines the desired joint variables required for motion control. Both problems involve estimating the kinematic parameters based on the pose of the end-effector. KI focuses on identification accuracy, whereas IK emphasizes fast convergence. Thus, they are usually studied independently. However, in specialized applications such as welding, assembly, and the 3C industry, robot kinematic parameters often vary due to thermal expansion, external loads, or mechanical wear, resulting in performance degradation. A promising solution is simultaneous online calibration and motion control [1]. This work aims to establish a unified framework for KI and IK and to develop an efficient, robust, and accurate solution method for both problems, which serves as a solid foundation for real-time identification and control.

The kinematic error model serves as the foundational step in solving KI and IK problems. The error model based on the product of exponentials (POE) formula has significant advantages over other models, making it more suitable for solving KI and IK problems.

1) *Continuity*: It preserves the smoothness property of the exponential mapping, ensuring that the error models are singularity-free.
2) *Completeness*: It completely corresponds any rigid-body motion to a specific twist according to Chasles' theory.
3) *Uniformity*: It can be uniformly applied to robots with any configuration, regardless of revolute and prismatic joints.

KI using the POE formula is typically solved by numerical methods. Park and Okamura [2] first developed the error models based on the global POE formula for KI. However, the formulation of the error models was not explicit since the differentials of the exponential map at joint twists were represented in a definite integral form. Subsequently, various improved methods were proposed. Chen *et al.* [3] introduced a local POE model for KI, which enables kinematic errors to exist only in the initial poses of consecutive local frames. He *et al.* [4] proposed an error model with explicit expressions based on the global POE formula and analyzed the identifiability of kinematic parameters. Chen *et al.* [5] employed the generalized spectral decomposition and partitioning operators to eliminate the redundant error parameters in [4] and to obtain the independent kinematic parameters. In addition, the relationship between the global POE and local POE formulas was analyzed, revealing no significant differences. Therefore, KI based on the global POE formula has recently been investigated [6, 7, 8, 9].

This work was supported by the National Key R&D Program of China (No. 2023YFB4706700). (Corresponding authors: Yu Sun and Xinyu Liu.)

Yuhan Chen, Yunkai Wang, Fan Bu, Changsheng Dai, and Xingjian Liu are with the Institute of Robotics and Intelligent Systems, Dalian University of Technology, Liaoning 116024, China (e-mail: yhchen@dlut.edu.cn; wyk1497988206@mail.dlut.edu.cn; 1499868503@mail.dlut.edu.cn; daichangsheng@dlut.edu.cn; xjliu@dlut.edu.cn).

Guiyang Xin is with the School of Biomedical Engineering, Dalian University of Technology, Liaoning 116024, China (e-mail: guiyang.xin@dlut.edu.cn).

Yu Sun is with the Department of Mechanical and Industrial Engineering, University of Toronto, Toronto, ON M5S 3G8, Canada (e-mail: sun@mie.utoronto.ca).

Xinyu Liu is with the Institute of Robotics and Intelligent Systems, Dalian University of Technology, Liaoning 116024, China, and also with the Department of Mechanical and Industrial Engineering, University of Toronto, Toronto, ON M5S 3G8, Canada (e-mail: xyliu@mie.utoronto.ca).

Li *et al.* [6] proposed an identification method based on joint axis configuration space and an adjoint error model, while Tao *et al.* [7] introduced an identification method using finite and instantaneous screw theory. Luo *et al.* [8] presented a novel parameter identification method that separately identifies the orientation error and position error parameters while uniformly updating the minimal parameter set. Huang *et al.* [9] reported a POE-based identification method using left-invariant error representation and decomposed iterative method.

Unfortunately, all the aforementioned KI methods are second-order iterative methods that depend on the Jacobian matrix, leading to limited identification accuracy. The Hessian matrix provides critical curvature information of the function, enabling the development of more accurate methods. Consequently, this work aims to propose a higher-order method based on the Hessian matrix for solving KI problems, enhancing identification accuracy.

IK has two types of solutions: analytical solutions and numerical solutions. The former is only available for serial robots whose configuration satisfies the Pieper criterion [10], while the latter is unaffected. Recent well-known analytical methods include those designed for 7-degrees-of-freedom (DOF) robots [11, 12], humanoid robots [13], and others. However, they are not general methods for solving IK. Existing numerical methods can be broadly categorized into heuristic-based, learning-based, optimization-based, and Jacobian-based methods. Heuristic-based methods [14, 15, 16] exhibit slow convergence when higher accuracy is required. Learning-based methods [17, 18, 19, 20] are primarily limited by inexact answers, data dependence, and lack of interpretability. Although optimization-based methods offer greater flexibility in handling redundancy resolution [21], Jacobian-based methods are less computationally intensive and easier to implement [22]. In addition, both types of methods share similarities, as they involve computing the differentials of the variables, including the damped least-squares method [23], selectively damped least squares method [24], Levenberg-Marquardt method [25], first-order Jacobian transpose techniques [26], and Jacobian pseudo-inverse methods [27].

Since all of these methods are first-order or second-order, there is potential for improving computational efficiency. Lloyd *et al.* [28] proposed a third-order iterative method called damped quick inverse kinematics (DQuIK), which is based on Halley's method. The rapid convergence of this method stems from utilizing not only the Jacobian but also the Hessian matrix. While the high-order method demonstrated strong performance in terms of convergence rate and robustness, both metrics can still be further enhanced for IK. Kou *et al.* [29] proposed a fifth-order method that fully exploits the Hessian matrix and exhibits better performance in scalar cases. Convergence rate and robustness are critical aspects in solving IK problems. Therefore, another motivation for this work is to investigate a higher-order method using the Hessian matrix to solve IK problems, improving both metrics.

In this paper, we present a fifth-order iterative method for solving the KI and IK problems of serial robots based on the global POE formula. The proposed method achieves high accuracy, rapid convergence, and strong robustness in addressing both problems. The main contributions of this paper are as follows:

1) We establish a general kinematic error model based on exponential coordinates and reformulate KI and IK as a root-finding problem.
2) We propose the modified Halley method to solve the above problem and prove that the method is of fifth-order convergence in matrix form.
3) We develop a damped modified Halley method to enhance the robustness of the original method.
4) We derive unified formulas for the Jacobian and Hessian matrices, essential for solving KI and IK problems, based on the time differential of exponential, with a simplified explicit version proposed for IK.
5) We fully implement the proposed method in MATLAB and C++ codebases and make the code available for general use.

The rest of the paper is organized in the following manner. Section II presents the computations required to solve robotic kinematic problems. Section III provides a general kinematic error model and proposes the modified Halley (MH) method to solve for KI and IK. The strategy to enhance the robustness of the MH method is also presented. Section IV describes the computation of the Jacobian and Hessian matrices required for the proposed method. Subsequently, section V validates our proposed method through various simulations and experiments. Finally, conclusions and future work are given in Section VI.

## II. Special Euclidean Group Theory

The Special Euclidean Group *SE*(3), the subgroup of the Lie group, can be used to describe rigid-body motion in robotics. A rigorous calculus computation can be constructed based on the Lie theory. In addition, exponential and logarithm mapping can transform between *SE*(3) and a six-dimensional tangent vector space, thus enabling elegant and rigorous solver designs for KI and IK. This section introduces the fundamental computations required for solving robot kinematics in subsequent analyses.

### A. Exponential Coordinates

For $\boldsymbol{T} \in SE(3)$, its exponential coordinates are $\boldsymbol{\lambda} = \left[\boldsymbol{\lambda}_\omega^T, \boldsymbol{\lambda}_\nu^T\right]^T \in \mathbb{R}^6$, where $\boldsymbol{\lambda}_\omega \in \mathbb{R}^3$ and $\boldsymbol{\lambda}_\nu \in \mathbb{R}^3$ are the direction and position vectors of $\boldsymbol{\lambda}$, respectively. The operators $(\cdot)^\wedge$ and $(\cdot)^\vee$ map a six-dimensional vector to/from a Lie algebra $\mathfrak{se}(3)$, and the operators $\lceil\cdot\rceil$ and $\lfloor\cdot\rfloor$ map a three-dimensional vector to/from a skew-symmetric matrix, such that

$$\boldsymbol{\lambda}^\wedge = \begin{bmatrix} \lceil\boldsymbol{\lambda}_\omega\rceil & \boldsymbol{\lambda}_\nu \\ \boldsymbol{0}^T & 0 \end{bmatrix} \in \mathfrak{se}(3), \qquad \left(\boldsymbol{\lambda}^\wedge\right)^\vee = \boldsymbol{\lambda}, \tag{1}$$

$$\lceil\boldsymbol{\lambda}_\omega\rceil = \begin{bmatrix} 0 & -\lambda_{\omega 3} & \lambda_{\omega 2} \\ \lambda_{\omega 3} & 0 & -\lambda_{\omega 1} \\ -\lambda_{\omega 2} & \lambda_{\omega 1} & 0 \end{bmatrix}, \quad \lfloor\lceil\boldsymbol{\lambda}_\omega\rceil\rfloor = \boldsymbol{\lambda}_\omega. \tag{2}$$

Note that the subscript number corresponds to an element's position in the vector.

The exponential map $\exp(\cdot)$ on $SE(3)$ coincides with the matrix exponential and has the closed-form expression [30]

$$\boldsymbol{T} = \exp(\boldsymbol{\lambda}^\wedge) = \begin{bmatrix} \boldsymbol{R} & \boldsymbol{p} \\ \mathbf{0}^T & 1 \end{bmatrix} = \begin{bmatrix} \exp(\lceil \boldsymbol{\lambda}_\omega \rceil) & \mathrm{dexp}_{\boldsymbol{\lambda}_\omega} \boldsymbol{\lambda}_\nu \\ \mathbf{0}^T & 1 \end{bmatrix}, \tag{3}$$

where

$$\exp(\lceil \boldsymbol{\lambda}_\omega \rceil) = \mathbf{I} + \alpha \lceil \boldsymbol{\lambda}_\omega \rceil + \frac{\beta}{2} \lceil \boldsymbol{\lambda}_\omega \rceil^2,$$
$$\mathrm{dexp}_{\boldsymbol{\lambda}_\omega} = \mathbf{I} + \frac{\beta}{2} \lceil \boldsymbol{\lambda}_\omega \rceil + \frac{1-\alpha}{\|\boldsymbol{\lambda}_\omega\|^2} \lceil \boldsymbol{\lambda}_\omega \rceil^2,$$

where $\mathbf{I}$ is the identity matrix, $\mathbf{0}$ represents the null vector or matrix, and $\|\cdot\|$ denotes the Euclidean norm,

$$\alpha = \frac{\sin(\|\boldsymbol{\lambda}_\omega\|/2)\cos(\|\boldsymbol{\lambda}_\omega\|/2)}{\|\boldsymbol{\lambda}_\omega\|/2}, \quad \beta = \left( \frac{\sin(\|\boldsymbol{\lambda}_\omega\|/2)}{\|\boldsymbol{\lambda}_\omega\|/2} \right)^2.$$

If $\|\boldsymbol{\lambda}_\omega\| \neq 2k\pi, k \in \mathbb{Z}$, then $\mathrm{dexp}_{\boldsymbol{\lambda}_\omega}$ is always invertible, and its inverse is written analytically as

$$\mathrm{dexp}^{-1}_{\boldsymbol{\lambda}_\omega} = \mathbf{I} - \frac{1}{2} \lceil \boldsymbol{\lambda}_\omega \rceil + \frac{1-\gamma}{\|\boldsymbol{\lambda}_\omega\|^2} \lceil \boldsymbol{\lambda}_\omega \rceil^2, \tag{4}$$

where $\gamma = \alpha/\beta$. If $\boldsymbol{\lambda}_\omega = \mathbf{0}$, then

$$\exp(\lceil \boldsymbol{\lambda}_\omega \rceil) = \mathrm{dexp}_{\boldsymbol{\lambda}_\omega} = \mathrm{dexp}^{-1}_{\boldsymbol{\lambda}_\omega} = \mathbf{I}. \tag{5}$$

The forward kinematics presented in Section III-A is computed based on (3).

Conversely, if $\boldsymbol{T}$ is known, $\boldsymbol{\lambda}$ can be computed from the matrix logarithm mapping $\log(\cdot)^\vee$ (see Appendix A for details). The kinematic error model proposed in Section III-B is constructed using $\log(\cdot)^\vee$.

### B. Time Differential of Exponential

Given $\boldsymbol{T}(t) \in SE(3)$, the time differential of the exponential map is formally defined as [31]

$$\dot{\boldsymbol{T}}(t) = \frac{\mathrm{d}}{\mathrm{d}t} \exp(\boldsymbol{\lambda}(t)^\wedge) = \exp(\boldsymbol{\lambda}(t)^\wedge) \left( \mathrm{dexp}(-\boldsymbol{\lambda}) \dot{\boldsymbol{\lambda}}(t) \right)^\wedge, \tag{6}$$

where

$$\mathrm{dexp}(\boldsymbol{\lambda}) = \begin{bmatrix} \mathrm{dexp}_{\boldsymbol{\lambda}_\omega} & \mathbf{0} \\ C_{\boldsymbol{\lambda}_\omega}(\boldsymbol{\lambda}_\nu) & \mathrm{dexp}_{\boldsymbol{\lambda}_\omega} \end{bmatrix},$$

where

$$C_{\boldsymbol{\lambda}_\omega}(\boldsymbol{y}) = \frac{\beta}{2} \lceil \boldsymbol{y} \rceil + \Lambda_1 \lceil \boldsymbol{y}, \boldsymbol{\lambda}_\omega \rceil + \Lambda_2 \boldsymbol{\lambda}_\omega^T \boldsymbol{y} \lceil \boldsymbol{\lambda}_\omega \rceil + \Lambda_3 \boldsymbol{\lambda}_\omega^T \boldsymbol{y} \lceil \boldsymbol{\lambda}_\omega \rceil^2,$$

where

$$\Lambda_1 = \frac{1-\alpha}{\|\boldsymbol{\lambda}_\omega\|^2}, \qquad \Lambda_2 = \frac{\alpha - \beta}{\|\boldsymbol{\lambda}_\omega\|^2},$$
$$\Lambda_3 = -\frac{1}{\|\boldsymbol{\lambda}_\omega\|^2} \left( \frac{3(1-\alpha)}{\|\boldsymbol{\lambda}_\omega\|^2} - \frac{\beta}{2} \right),$$

and we define $\lceil \boldsymbol{a}, \boldsymbol{b} \rceil = \lceil \boldsymbol{a} \rceil \lceil \boldsymbol{b} \rceil + \lceil \boldsymbol{b} \rceil \lceil \boldsymbol{a} \rceil$ for simplicity. If $\|\boldsymbol{\lambda}_\omega\| \neq 2k\pi, k \in \mathbb{Z}$, then $\mathrm{dexp}(\boldsymbol{\lambda})$ is invertible, and its inverse is expressed analytically as

$$\mathrm{dexp}^{-1}(\boldsymbol{\lambda}) = \begin{bmatrix} \mathrm{dexp}^{-1}_{\boldsymbol{\lambda}_\omega} & \mathbf{0} \\ D_{\boldsymbol{\lambda}_\omega}(\boldsymbol{\lambda}_\nu) & \mathrm{dexp}^{-1}_{\boldsymbol{\lambda}_\omega} \end{bmatrix}, \tag{7}$$

where

$$D_{\boldsymbol{\lambda}_\omega}(\boldsymbol{y}) = -\frac{1}{2} \lceil \boldsymbol{y} \rceil + \Lambda_4 \lceil \boldsymbol{y}, \boldsymbol{\lambda}_\omega \rceil + \Lambda_5 \boldsymbol{\lambda}_\omega^T \boldsymbol{y} \lceil \boldsymbol{\lambda}_\omega \rceil^2,$$

$$\begin{tikzcd}[column sep=huge, row sep=large]
\lambda \in \mathbb{R}^6 \arrow[r, shift left, "\exp(\cdot^\wedge)"] \arrow[d, "\frac{\mathrm{d}}{\mathrm{d}t}"'] & \mathbf{T} \in SE(3) \arrow[l, shift left, "\log(\cdot)^\vee"] \arrow[d, "\left(\mathbf{T}^{-1}\dot{\mathbf{T}}\right)^\vee"] \\
\dot{\lambda} \in \mathbb{R}^6 \arrow[r, shift left, "\mathrm{dexp}_{-\lambda}"] & V \in \mathbb{R}^6 \arrow[l, shift left, "\mathrm{dexp}^{-1}_{-\lambda}"]
\end{tikzcd}$$

Fig. 1. Transformation between $SE(3)$ and exponential coordinates.

where

$$\Lambda_4 = \frac{1-\gamma}{\|\boldsymbol{\lambda}_\omega\|^2}, \qquad \Lambda_5 = \frac{1/\beta + \gamma - 2}{\|\boldsymbol{\lambda}_\omega\|^4}.$$

The body twist $\boldsymbol{V} = \begin{bmatrix} \boldsymbol{\omega}^T & \boldsymbol{\nu}^T \end{bmatrix}^T \in \mathbb{R}^6$, where $\boldsymbol{\omega} \in \mathbb{R}^3$ and $\boldsymbol{\nu} \in \mathbb{R}^3$ are angular and linear velocities, can be computed by

$$\boldsymbol{V}(t)^\wedge = \boldsymbol{T}^{-1}(t) \dot{\boldsymbol{T}}(t). \tag{8}$$

Based on (3) and (6), we can get

$$\boldsymbol{V}(t) = \mathrm{dexp}(-\boldsymbol{\lambda}) \dot{\boldsymbol{\lambda}}(t). \tag{9}$$

Provided that $\mathrm{dexp}(-\boldsymbol{\lambda})$ is non-singular, we have

$$\dot{\boldsymbol{\lambda}}(t) = \mathrm{dexp}^{-1}(-\boldsymbol{\lambda}) \boldsymbol{V}(t). \tag{10}$$

The Jacobian matrix computed in Section IV-A is based on (8).

The transformation between $SE(3)$ and exponential coordinates is summarized in Fig. 1.

### C. Adjoint Representation

The adjoint representation of $\boldsymbol{T}$ is

$$\mathrm{Ad}_{\boldsymbol{T}} = \begin{bmatrix} \boldsymbol{R} & \mathbf{0} \\ \lceil \boldsymbol{p} \rceil \boldsymbol{R} & \boldsymbol{R} \end{bmatrix} \in \mathbb{R}^{6 \times 6}, \tag{11}$$

and its inverse is written analytically as

$$\mathrm{Ad}^{-1}_{\boldsymbol{T}} = \mathrm{Ad}_{\boldsymbol{T}^{-1}} = \begin{bmatrix} \boldsymbol{R}^T & \mathbf{0} \\ -\boldsymbol{R}^T \lceil \boldsymbol{p} \rceil & \boldsymbol{R}^T \end{bmatrix} \in \mathbb{R}^{6 \times 6}. \tag{12}$$

The adjoint map possesses the following properties, verifiable by direct computation:

$$\boldsymbol{T} \boldsymbol{\lambda}^\wedge \boldsymbol{T}^{-1} = \left( \mathrm{Ad}_{\boldsymbol{T}} \boldsymbol{\lambda} \right)^\wedge, \tag{13}$$
$$\mathrm{Ad}_{\boldsymbol{T}_1} \mathrm{Ad}_{\boldsymbol{T}_2} \boldsymbol{\lambda} = \mathrm{Ad}_{\boldsymbol{T}_1 \boldsymbol{T}_2} \boldsymbol{\lambda}. \tag{14}$$

The Jacobian matrix derived in Section IV-A is founded on (11)-(14).

The time differential of $\mathrm{Ad}_{\boldsymbol{T}}$ satisfies

$$\frac{\mathrm{d}}{\mathrm{d}t} \mathrm{Ad}_{\boldsymbol{T}} = \mathrm{Ad}_{\boldsymbol{T}} \, \mathrm{ad}(\boldsymbol{V}), \tag{15}$$

where $\mathrm{ad}(\boldsymbol{V})$ is the adjoint operator and is defined by

$$\mathrm{ad}(\boldsymbol{V}) = \begin{bmatrix} \lceil \boldsymbol{\omega} \rceil & \mathbf{0} \\ \lceil \boldsymbol{\nu} \rceil & \lceil \boldsymbol{\omega} \rceil \end{bmatrix} \in \mathbb{R}^{6 \times 6}.$$

The Hessian matrix derived in Section IV-B is based on (15).

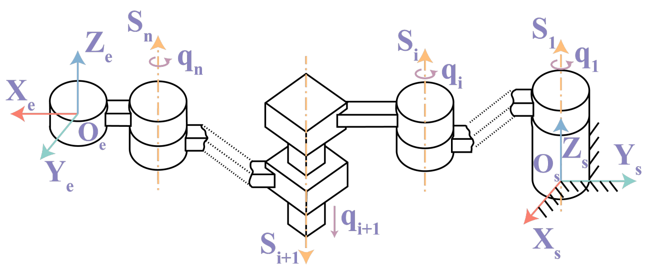



Fig. 2. POE formula for an $n$-DOF serial robot.

## III. Error Modeling and Parameter Solving

This section introduces a kinematic error model and reformulates KI and IK as a root-finding problem. Subsequently, the modified Halley's method is devised to solve the above problem. The strategy to enhance the robustness of the proposed method is also presented.

### A. Forward Kinematics using POE Formula

To use the global POE formula, it is sufficient to assign a stationary reference frame $\{s\}$ (e.g., fixed at the robot's base) and a frame $\{e\}$ at the end-effector, as shown in Fig. 2. $\boldsymbol{\Gamma} \in \mathbb{R}^6$ describes the exponential coordinates of the end-effector pose relative to the frame $\{s\}$ when the robot is at its zero position. A twist $\boldsymbol{S}_i \in \mathbb{R}^6$ expressed in the frame $\{s\}$ is associated with the $i$th joint motions. In addition, $q_i$, the actual value of the joint variable, is written as $q_i = \bar{q}_i + \delta q_i$, where $\bar{q}_i$ is the nominal value of the joint variable, and $\delta q_i$ is the joint zero-position errors.

The POE formula describing the forward kinematics of an $n$-DOF serial robot follows

$$\boldsymbol{T}_e = \prod_{i=1}^{n} \boldsymbol{T}_i \boldsymbol{T}_{n+1}, \tag{16}$$

where

$$\begin{aligned} \boldsymbol{T}_i &= \exp\left(\boldsymbol{S}_i^{\wedge}(\bar{q}_i + \delta q_i)\right), \\ \boldsymbol{T}_{n+1} &= \exp\left(\boldsymbol{\Gamma}^{\wedge}\right). \end{aligned}$$

Therefore, the elements required for the POE formula include $\boldsymbol{S}$, $\boldsymbol{\Gamma}$, and $\boldsymbol{q}$, where

$$\boldsymbol{S} = \begin{bmatrix} \boldsymbol{S}_1^T & \boldsymbol{S}_2^T & \cdots & \boldsymbol{S}_n^T \end{bmatrix}^T \in \mathbb{R}^{6n}, \tag{17}$$

$$\boldsymbol{q} = \bar{\boldsymbol{q}} + \delta\boldsymbol{q}, \tag{18}$$

where

$$\begin{aligned} \bar{\boldsymbol{q}} &= \begin{bmatrix} \bar{q}_1 & \bar{q}_2 & \cdots & \bar{q}_n \end{bmatrix}^T \in \mathbb{R}^n, \\ \delta\boldsymbol{q} &= \begin{bmatrix} \delta q_1 & \delta q_2 & \cdots & \delta q_n \end{bmatrix}^T \in \mathbb{R}^n. \end{aligned}$$

### B. Kinematic Error Model

Robotic kinematic problems all focus on (16). Assuming that $\boldsymbol{S}$, $\boldsymbol{\Gamma}$, and $\delta\boldsymbol{q}$ are variables, a general kinematic error model can be derived. It should be noted that the proposed method does not consider the pose error of the end-effector caused by the deformation of the robot's joints and links.

Provided the desired end-effector pose $\boldsymbol{T}_d \in SE(3)$, the kinematic error based on exponential coordinates is defined by

$$\boldsymbol{\xi}(\boldsymbol{S}, \boldsymbol{\Gamma}, \delta\boldsymbol{q}) = \log\left(\boldsymbol{T}_\xi\right)^{\vee}, \quad \text{with} \quad \boldsymbol{T}_\xi = \boldsymbol{T}_e^{-1}\boldsymbol{T}_d. \tag{19}$$

Based on (19), KI and IK can be reformulated as root-finding problems, i.e., solving $\boldsymbol{\Phi}^*$ so that it satisfies

$$\boldsymbol{\Xi}(\boldsymbol{\Phi}^*) = \boldsymbol{0}, \tag{20}$$

where $\boldsymbol{\Phi}^*$ is the desired value of $\boldsymbol{\Phi}$ and $\boldsymbol{\Xi}$ is a problem-specific error.

*1) Kinematic Identification:* KI involves determining the solution parameters $(\boldsymbol{S}^*, \boldsymbol{\Gamma}^*, \delta\boldsymbol{q}^*)$ by satisfying (20), given $m$ sets of known measurement data $(\bar{\boldsymbol{q}}_1, \cdots, \bar{\boldsymbol{q}}_m, \boldsymbol{T}_{d_1}, \cdots, \boldsymbol{T}_{d_m})$. Hence, $\boldsymbol{\Phi}$ and $\boldsymbol{\Xi}$ are defined as

$$\boldsymbol{\Phi} = \begin{bmatrix} \boldsymbol{S}^T & \boldsymbol{\Gamma}^T & \delta\boldsymbol{q}^T \end{bmatrix}^T \in \mathbb{R}^{7n+6}, \tag{21}$$

$$\boldsymbol{\Xi}(\boldsymbol{\Phi}) = \begin{bmatrix} \boldsymbol{\xi}_1^T(\boldsymbol{\Phi}) & \boldsymbol{\xi}_2^T(\boldsymbol{\Phi}) & \cdots & \boldsymbol{\xi}_m^T(\boldsymbol{\Phi}) \end{bmatrix}^T \in \mathbb{R}^{6m}, \tag{22}$$

where $\boldsymbol{\xi}_i$ can be computed by (19). Specifically,

$$\boldsymbol{\xi}_i(\boldsymbol{\Phi}) = \log\left(\boldsymbol{T}_{\xi_i}\right)^{\vee}, \quad \text{with} \quad \boldsymbol{T}_{\xi_i} = \boldsymbol{T}_{e_i}^{-1}(\boldsymbol{\Phi}, \bar{\boldsymbol{q}}_i)\boldsymbol{T}_{d_i}, \tag{23}$$

where $\boldsymbol{T}_{e_i}(\boldsymbol{\Phi}, \bar{\boldsymbol{q}}_i)$ is expressed in (16). It is worth noting that although $\boldsymbol{\Phi}$ contains $7n+6$ elements, only $6r+3t+6$ elements can be identified [4], where $r$ represents the number of revolute joints, and $t$ denotes the number of prismatic joints. The proposed error model is designed to unify KI and IK within a consistent framework. Consequently, it is not formulated as a minimal kinematic model, and dependencies exist among the identified parameters (see [32, 33, 34] for details).

*2) Inverse Kinematics:* IK involves determining the solution $\delta\boldsymbol{q}^*$ of (20), given the desired end-effector pose $\boldsymbol{T}_{d_1}$ (i.e. $m = 1$) and known constant parameters $\boldsymbol{S}, \boldsymbol{\Gamma}$, and $\bar{\boldsymbol{q}}_1 = \boldsymbol{0}$. Therefore, $\boldsymbol{\Phi}$ and $\boldsymbol{\Xi}$ are written as

$$\boldsymbol{\Phi} = \delta\boldsymbol{q} \in \mathbb{R}^n, \tag{24}$$

$$\boldsymbol{\Xi}(\boldsymbol{\Phi}) = \boldsymbol{\xi}_1(\boldsymbol{\Phi}) \in \mathbb{R}^6. \tag{25}$$

### C. Modified Halley Method for Solving KI and IK

For the root-finding problems in (20), a general iterative computation is expressed as

$$\boldsymbol{\Phi}^{(\tau+1)} = \boldsymbol{\Phi}^{(\tau)} + \Delta\boldsymbol{\Phi}^{(\tau)}, \tag{26}$$

where the superscript indicates the iteration number. Various methods employ distinct strategies for computing $\Delta\boldsymbol{\Phi}$.

The first-order and second-order Taylor polynomials of (20) are written as

$$\boldsymbol{\Xi}(\boldsymbol{\Phi} + \Delta\boldsymbol{\Phi}) \approx \boldsymbol{\Xi}(\boldsymbol{\Phi}) + \boldsymbol{J}_{\Xi}\Delta\boldsymbol{\Phi}, \tag{27}$$

$$\boldsymbol{\Xi}(\boldsymbol{\Phi} + \Delta\boldsymbol{\Phi}) \approx \boldsymbol{\Xi}(\boldsymbol{\Phi}) + \boldsymbol{J}_{\Xi}\Delta\boldsymbol{\Phi} + \frac{1}{2}\boldsymbol{H}_{\Xi}\left[\Delta\boldsymbol{\Phi}\right]\Delta\boldsymbol{\Phi}, \tag{28}$$

where the second derivative term $\boldsymbol{H}_{\Xi} \in \mathbb{R}^{l_\Xi \times l_\Phi \times l_\Phi}$ is a rank-3 tensor, Jacobian matrix $\boldsymbol{J}_{\Xi} \in \mathbb{R}^{l_\Xi \times l_\Phi}$ is defined as

$$\boldsymbol{J}_{\Xi} = \begin{bmatrix} \frac{\partial \Xi_1}{\partial \Phi_1} & \frac{\partial \Xi_1}{\partial \Phi_2} & \cdots & \frac{\partial \Xi_1}{\partial \Phi_{l_\Phi}} \\ \frac{\partial \Xi_2}{\partial \Phi_1} & \frac{\partial \Xi_2}{\partial \Phi_2} & \cdots & \frac{\partial \Xi_2}{\partial \Phi_{l_\Phi}} \\ \vdots & \vdots & \ddots & \vdots \\ \frac{\partial \Xi_{l_\Xi}}{\partial \Phi_1} & \frac{\partial \Xi_{l_\Xi}}{\partial \Phi_2} & \cdots & \frac{\partial \Xi_{l_\Xi}}{\partial \Phi_{l_\Phi}} \end{bmatrix},$$

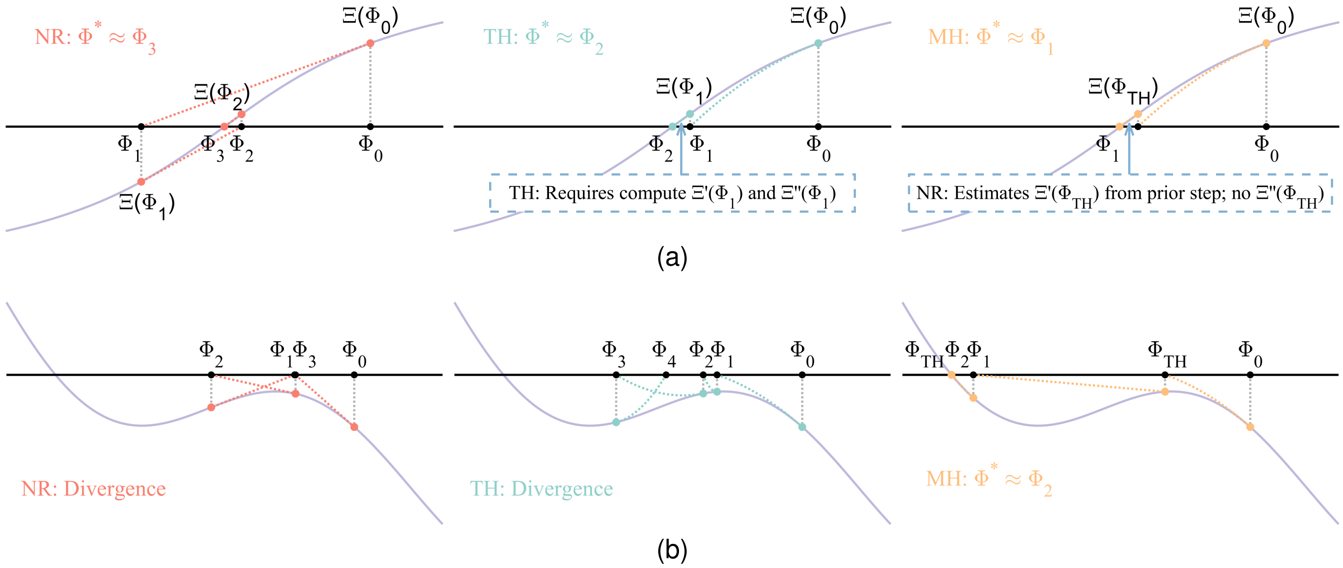



Fig. 3. Graphical illustration of the NR, TH, and MH methods on one-dimensional functions. (a) $\Xi(\Phi) = \arctan(\Phi)$. (b) $\Xi(\Phi) = \sin(\Phi) + 0.5\cos(3\Phi) - 0.1\Phi$. The MH method achieves the fewest iterations (top), while the NR and TH methods diverge, with only the MH method converging (bottom). In the latter case, the $\Xi'(\Phi_{\mathrm{TH}})$ estimated by $\Xi''(\Phi_0)$ has the opposite sign of the true $\Xi'(\Phi_{\mathrm{TH}})$, which facilitates convergence.

and the tensor contraction $\boldsymbol{H}_\Xi[\Delta\boldsymbol{\Phi}] \in \mathbb{R}^{l_\Xi \times l_\Phi}$ can be written as

$$\boldsymbol{H}_\Xi[\Delta\boldsymbol{\Phi}] = \sum_{i=1}^{l_\Phi} \boldsymbol{H}_{\Xi_i}\Delta\Phi_i,$$

where $\Delta\Phi_i$ is the $i$th element of $\Delta\boldsymbol{\Phi}$ and the kinematic Hessian matrix $\boldsymbol{H}_{\Xi_i} \in \mathbb{R}^{l_\Xi \times l_\Phi}$ is defined as

$$\boldsymbol{H}_{\Xi_i} = \frac{\partial \boldsymbol{J}_\Xi}{\partial \Phi_i} = \begin{bmatrix} \frac{\partial^2 \Xi_1}{\partial \Phi_1 \partial \Phi_i} & \frac{\partial^2 \Xi_1}{\partial \Phi_2 \partial \Phi_i} & \cdots & \frac{\partial^2 \Xi_1}{\partial \Phi_{l_\Phi} \partial \Phi_i} \\ \frac{\partial^2 \Xi_2}{\partial \Phi_1 \partial \Phi_i} & \frac{\partial^2 \Xi_2}{\partial \Phi_2 \partial \Phi_i} & \cdots & \frac{\partial^2 \Xi_2}{\partial \Phi_{l_\Phi} \partial \Phi_i} \\ \vdots & \vdots & \ddots & \vdots \\ \frac{\partial^2 \Xi_{l_\Xi}}{\partial \Phi_1 \partial \Phi_i} & \frac{\partial^2 \Xi_{l_\Xi}}{\partial \Phi_2 \partial \Phi_i} & \cdots & \frac{\partial^2 \Xi_{l_\Xi}}{\partial \Phi_{l_\Phi} \partial \Phi_i} \end{bmatrix},$$

where $l_\Xi$ and $l_\Phi$ are the length of $\boldsymbol{\Xi}$ and $\boldsymbol{\Phi}$, respectively. The traditional Halley (TH) method for KI and IK aims to solve for $\Delta\boldsymbol{\Phi}$ given $\boldsymbol{\Phi}$ such that (28) satisfies $\boldsymbol{\Xi}(\boldsymbol{\Phi} + \Delta\boldsymbol{\Phi}) = \boldsymbol{0}$.

It is challenging to solve the multidimensional quadratic equations. Instead, $\Delta\boldsymbol{\Phi}$ can be pre-estimated by (27) as

$$\Delta\boldsymbol{\Phi}_{\mathrm{NR}} = -\boldsymbol{J}_\Xi^\dagger \boldsymbol{\Xi}, \tag{29}$$

where $(\cdot)^\dagger$ is the left inverse, defined as

$$\boldsymbol{A}^\dagger = (\boldsymbol{A}^T\boldsymbol{A})^{-1}\boldsymbol{A}^T. \tag{30}$$

Note that (29) is the update strategy for the Newton–Raphson (NR) method, which is a second-order root-finding method [35]. Substituted (29) into (28), we have

$$\boldsymbol{\Xi}(\boldsymbol{\Phi}) + \boldsymbol{J}_\Xi\Delta\boldsymbol{\Phi} + \frac{1}{2}\boldsymbol{H}_\Xi[\Delta\boldsymbol{\Phi}_{\mathrm{NR}}]\Delta\boldsymbol{\Phi} = \boldsymbol{0}. \tag{31}$$

Hence, (31) can be rearranged for $\Delta\boldsymbol{\Phi}$ and yields the update strategy for the TH method

$$\Delta\boldsymbol{\Phi}_{\mathrm{TH}} = -\boldsymbol{J}_{\Xi_{\mathrm{NR}}}^\dagger \boldsymbol{\Xi}, \tag{32}$$

where

$$\boldsymbol{J}_{\Xi_{\mathrm{NR}}} = \boldsymbol{J}_\Xi + \frac{1}{2}\boldsymbol{H}_\Xi[\Delta\boldsymbol{\Phi}_{\mathrm{NR}}].$$

The TH method is the third-order root-finding method [35].

Given the computational complexity of $\boldsymbol{H}_{\Xi_i}$ in each iteration, maximizing its utilization is beneficial for effectively solving the problem. According to (26), the parameter estimated by the TH method can be expressed as

$$\boldsymbol{\Phi}_{\mathrm{TH}} = \boldsymbol{\Phi} + \Delta\boldsymbol{\Phi}_{\mathrm{TH}}. \tag{33}$$

Based on (22) or (25), we can estimate the error of the next step, yielding

$$\boldsymbol{\Xi}_{\mathrm{TH}} = \boldsymbol{\Xi}(\boldsymbol{\Phi}_{\mathrm{TH}}). \tag{34}$$

The first-order Taylor polynomials of (20) about $\boldsymbol{\Phi}_{\mathrm{TH}}$ are written as

$$\boldsymbol{\Xi}(\boldsymbol{\Phi}_{\mathrm{TH}} + \Delta\boldsymbol{\Phi}) \approx \boldsymbol{\Xi}(\boldsymbol{\Phi}_{\mathrm{TH}}) + \boldsymbol{J}_{\Xi_{\mathrm{TH}}}\Delta\boldsymbol{\Phi}, \tag{35}$$

where $\boldsymbol{J}_{\Xi_{\mathrm{TH}}}$ can be estimated as

$$\boldsymbol{J}_{\Xi_{\mathrm{TH}}} = \boldsymbol{J}_\Xi + \boldsymbol{H}_\Xi[\Delta\boldsymbol{\Phi}_{\mathrm{TH}}].$$

To satisfy $\boldsymbol{\Xi}(\boldsymbol{\Phi}_{\mathrm{TH}} + \Delta\boldsymbol{\Phi}) = \boldsymbol{0}$, the update strategy for the proposed MH method is

$$\Delta\boldsymbol{\Phi}_{\mathrm{MH}} = -\boldsymbol{J}_{\Xi_{\mathrm{TH}}}^\dagger \boldsymbol{\Xi}_{\mathrm{TH}}. \tag{36}$$

Finally, the iterative formula of the MH method is

$$\boldsymbol{\Phi}^{(\tau+1)} = \boldsymbol{\Phi}_{\mathrm{TH}} + \Delta\boldsymbol{\Phi}_{\mathrm{MH}}. \tag{37}$$

Fig. 3 provides a graphical illustration of the NR, TH, and MH methods on one-dimensional functions, offering an intuitive understanding of the MH method. Note that in the MH method, the kinematic Hessian matrices remain unchanged in the second sub-step, which is considered part of a single iteration. It can be observed that the proposed MH method only adds one evaluation of the error at another point iterated by the TH methods. Still, its order of convergence will be proven to increase from three to five in Appendix B, which represents one of our theoretical contributions. The superiority of the MH method over the TH method is demonstrated in Section V-B.

### D. Damped Modified Halley Method

The HM method requires computing the left inverse of the matrix. When the rank of $\boldsymbol{A}$ in (30) is not appropriately related to its dimensions, the computation of its left inverse may lead to singularity. Specifically, if the rank of $\boldsymbol{J}_{\Xi}$ is $r$ and $r < l_{\Phi}$, then $\boldsymbol{J}_{\Xi}^T \boldsymbol{J}_{\Xi} \in \mathbb{R}^{l_\Phi \times l_\Phi}$ is singular, leading to the failure of the root-finding process. Among the various methods for handling the singular problem, one of the most common is to incorporate a damping term into the computational process. In optimization, this strategy is also known as the Levenberg–Marquardt method [36]. Therefore, the damped Newton–Raphson (DNR) method [23] and damped traditional Halley (DTH) method [28] were developed.

Inspired by them, we propose the damped modified Halley (DMH) method to improve robustness, introducing the damping factor $\sigma$ into the left inverse in (29), (32), and (36). First, the update strategy for the DNR method can be obtained by

$$\Delta\boldsymbol{\Phi}_{\mathrm{DNR}} = -(\boldsymbol{J}_{\Xi}^T \boldsymbol{J}_{\Xi} + \sigma \mathbf{I})^{-1} \boldsymbol{J}_{\Xi}^T \boldsymbol{\Xi}. \tag{38}$$

Then, we have the update strategy for the DTH method

$$\Delta\boldsymbol{\Phi}_{\mathrm{DTH}} = -(\boldsymbol{J}_{\Xi_{\mathrm{DNR}}}^T \boldsymbol{J}_{\Xi_{\mathrm{DNR}}} + \sigma \mathbf{I})^{-1} \boldsymbol{J}_{\Xi_{\mathrm{DNR}}}^T \boldsymbol{\Xi}, \tag{39}$$

where

$$\boldsymbol{J}_{\Xi_{\mathrm{DNR}}} = \boldsymbol{J}_{\Xi} + \frac{1}{2}\boldsymbol{H}_{\Xi}\left[\Delta\boldsymbol{\Phi}_{\mathrm{DNR}}\right].$$

Next, the error of the next step can be estimated by

$$\boldsymbol{\Xi}_{\mathrm{DTH}} = \boldsymbol{\Xi}(\boldsymbol{\Phi}_{\mathrm{DTH}}), \tag{40}$$

where

$$\boldsymbol{\Phi}_{\mathrm{DTH}} = \boldsymbol{\Phi} + \Delta\boldsymbol{\Phi}_{\mathrm{DTH}}.$$

Finally, the update strategy for the DMH method is expressed as

$$\Delta\boldsymbol{\Phi}_{\mathrm{DMH}} = -(\boldsymbol{J}_{\Xi_{\mathrm{DTH}}}^T \boldsymbol{J}_{\Xi_{\mathrm{DTH}}} + \sigma \mathbf{I})^{-1} \boldsymbol{J}_{\Xi_{\mathrm{DTH}}}^T \boldsymbol{\Xi}_{\mathrm{DTH}}, \tag{41}$$

where

$$\boldsymbol{J}_{\Xi_{\mathrm{DTH}}} = \boldsymbol{J}_{\Xi} + \boldsymbol{H}_{\Xi}\left[\Delta\boldsymbol{\Phi}_{\mathrm{DTH}}\right].$$

The iterative formula of the DMH method is

$$\boldsymbol{\Phi}^{(\tau+1)} = \boldsymbol{\Phi}_{\mathrm{DTH}} + \Delta\boldsymbol{\Phi}_{\mathrm{DMH}}. \tag{42}$$

It is essential to note that the effectiveness of all damping methods relies on the proper selection of the damping factor $\sigma$, which involves a trade-off between convergence efficiency and robustness. If $\sigma$ is too small, the method may experience issues near singularities; conversely, an excessively large $\sigma$ may overly warp the iteration step, resulting in slower convergence. The selection of $\sigma$ is not the primary focus of the current work; therefore, we empirically set it as a constant $\sigma = 10^{-6}$. Fortunately, a substantial body of literature exists on this subject. Existing methods [24, 25, 37] for adaptive damping in the DNR framework can be equally extended to the proposed method.

## IV. Derivation of Jacobian Matrix and Hessian Matrix

Using the proposed method, the key lies in computing $\boldsymbol{J}_{\Xi}$ and $\boldsymbol{H}_{\Xi}$ in (27) and (28). Based on (22) and (25), $\boldsymbol{J}_{\Xi}$ can also be represented as

$$\boldsymbol{J}_{\Xi} = \begin{bmatrix} \boldsymbol{J}_{\xi_1}^T & \boldsymbol{J}_{\xi_2}^T & \cdots & \boldsymbol{J}_{\xi_m}^T \end{bmatrix}^T \tag{43}$$

and $\boldsymbol{H}_{\Xi}$ depends on $\boldsymbol{H}_{\Xi_i}$. For the IK problem, the parameter $m$ is set to 1. Therefore, this section describes how to compute $\boldsymbol{J}_{\xi_i}$ and $\boldsymbol{H}_{\Xi_i}$.

### A. Computation of Jacobian Matrix $\boldsymbol{J}_{\xi_i}$

For simplicity, we drop the subscript $i$. The time differential of $\boldsymbol{\xi}$ can be obtained by

$$\dot{\boldsymbol{\xi}} = \boldsymbol{J}_{\xi}\dot{\boldsymbol{\Phi}}, \quad \text{with} \quad \boldsymbol{J}_{\xi} = \frac{\partial \boldsymbol{\xi}}{\partial \boldsymbol{\Phi}}, \tag{44}$$

where $\boldsymbol{\Phi}$ is expressed as (21) and (24) for KI and IK, respectively.

Since $\boldsymbol{T}_d$ is a constant and based on (8) and (13), the twist $\boldsymbol{V}_{\xi}$ associated with $\boldsymbol{T}_{\xi}$ in (23) is defined by

$$\begin{aligned}
\boldsymbol{V}_{\xi}^{\wedge} &= \boldsymbol{T}_{\xi}^{-1}\dot{\boldsymbol{T}}_{\xi} = \boldsymbol{T}_{\xi}^{-1}\left(\frac{\mathrm{d}}{\mathrm{d}t}\left(\boldsymbol{T}_e^{-1}\right)\boldsymbol{T}_d\right) \\
&= \boldsymbol{T}_{\xi}^{-1}\left(-\boldsymbol{T}_e^{-1}\dot{\boldsymbol{T}}_e\boldsymbol{T}_e^{-1}\boldsymbol{T}_d\right) = -\boldsymbol{T}_{\xi}^{-1}\boldsymbol{V}_e^{\wedge}\boldsymbol{T}_{\xi} \\
&= -\left(\mathrm{Ad}_{\boldsymbol{T}_{\xi}}^{-1}\boldsymbol{V}_e\right)^{\wedge},
\end{aligned}$$

which yields

$$\boldsymbol{V}_{\xi} = -\mathrm{Ad}_{\boldsymbol{T}_{\xi}}^{-1}\boldsymbol{V}_e, \tag{45}$$

where $\boldsymbol{V}_e$ is a body twist associated with $\boldsymbol{T}_e$. Combined with (10) and (45), we have

$$\begin{aligned}
\dot{\boldsymbol{\xi}} &= -\mathrm{dexp}^{-1}(-\boldsymbol{\xi})\mathrm{Ad}_{\boldsymbol{T}_{\xi}}^{-1}\boldsymbol{V}_e \\
&= -\mathrm{dexp}^{-1}(\boldsymbol{\xi})\boldsymbol{V}_e.
\end{aligned} \tag{46}$$

According to (8), we have

$$\boldsymbol{V}_e^{\wedge} = \boldsymbol{T}_e^{-1}\dot{\boldsymbol{T}}_e. \tag{47}$$

Injecting (16) into (47), we get

$$\begin{aligned}
\boldsymbol{V}_e^{\wedge} &= \left(\prod_{i=1}^{n}\boldsymbol{T}_i\boldsymbol{T}_{n+1}\right)^{-1}\left(\sum_{i=1}^{n+1}\prod_{j=1}^{i-1}\boldsymbol{T}_j\dot{\boldsymbol{T}}_i\prod_{k=i+1}^{n+1}\boldsymbol{T}_k\right) \\
&= \sum_{i=1}^{n+1}\boldsymbol{T}_{i\sim n}^{-1}\boldsymbol{T}_i^{-1}\dot{\boldsymbol{T}}_i\boldsymbol{T}_{i\sim n} \\
&= \sum_{i=1}^{n+1}\left(\mathrm{Ad}_{\boldsymbol{T}_{i\sim n}}^{-1}\boldsymbol{V}_i\right)^{\wedge},
\end{aligned} \tag{48}$$

where

$$\boldsymbol{T}_{i\sim n} = \prod_{j=i+1}^{n+1}\boldsymbol{T}_j, \tag{49}$$

$\boldsymbol{V}_i$ is a body twist associated with $\boldsymbol{T}_i$ and can be given by

$$\boldsymbol{V}_i = \begin{cases} \mathrm{dexp}\left(-\boldsymbol{S}_i(\bar{q}_i + \delta q_i)\right) & \\ \qquad \left(\dot{\boldsymbol{S}}_i(\bar{q}_i + \delta q_i) + \boldsymbol{S}_i\delta\dot{q}_i\right), & i = 1, \cdots, n \\ \mathrm{dexp}(-\boldsymbol{\Gamma})\dot{\boldsymbol{\Gamma}}, & i = n+1. \end{cases}$$

Based on (48), $\boldsymbol{V}_e$ can be represented in matrix form

$$\boldsymbol{V}_e = \boldsymbol{J}_\Phi \dot{\boldsymbol{\Phi}}, \tag{50}$$

where

$$\boldsymbol{J}_\Phi = \begin{cases} \begin{bmatrix} \boldsymbol{J}_{\boldsymbol{S}} & \boldsymbol{J}_\Gamma & \boldsymbol{J}_{\delta \boldsymbol{q}} \end{bmatrix} & \text{for KI,} \\ \boldsymbol{J}_{\delta \boldsymbol{q}} & \text{for IK,} \end{cases} \tag{51}$$

where

$$\begin{aligned} \boldsymbol{J}_{\boldsymbol{S}} &= \begin{bmatrix} \boldsymbol{J}_{S_1} & \boldsymbol{J}_{S_2} & \cdots & \boldsymbol{J}_{S_n} \end{bmatrix} \in \mathbb{R}^{6\times 6n}, \\ \boldsymbol{J}_\Gamma &= \mathrm{dexp}(-\boldsymbol{\Gamma}) \in \mathbb{R}^{6\times 6}, \\ \boldsymbol{J}_{\delta \boldsymbol{q}} &= \begin{bmatrix} \boldsymbol{J}_{\delta q_1} & \boldsymbol{J}_{\delta q_2} & \cdots & \boldsymbol{J}_{\delta q_n} \end{bmatrix} \in \mathbb{R}^{6\times n}, \end{aligned}$$

where $\boldsymbol{J}_{S_i} \in \mathbb{R}^{6\times 6}$ and $\boldsymbol{J}_{\delta q_i} \in \mathbb{R}^{6\times 1}$ are written as

$$\begin{aligned} \boldsymbol{J}_{S_i} &= \mathrm{Ad}^{-1}_{\boldsymbol{T}_{i\sim n}} \mathrm{dexp}\left(-\boldsymbol{S}_i(\bar{q}_i + \delta q_i)\right)(\bar{q}_i + \delta q_i), \\ \boldsymbol{J}_{\delta q_i} &= \mathrm{Ad}^{-1}_{\boldsymbol{T}_{i\sim n}} \mathrm{dexp}\left(-\boldsymbol{S}_i(\bar{q}_i + \delta q_i)\right)\boldsymbol{S}_i \\ &= \mathrm{Ad}^{-1}_{\boldsymbol{T}_{i\sim n}} \boldsymbol{S}_i. \end{aligned} \tag{52}$$

Combined with (44), (46), and (50), $\boldsymbol{J}_\xi$ is finally summarized as

$$\boldsymbol{J}_\xi = -\mathrm{dexp}^{-1}(\boldsymbol{\xi})\boldsymbol{J}_\Phi. \tag{53}$$

Different from the error model proposed by He *et al.* [4], which relies on a first-order approximation, our approach makes no such approximation. This purely analytical treatment ensures higher rigor in derivation.

### B. Computation of Kinematic Hessian Matrix$\boldsymbol{H}_{\Xi_i}$

The kinematic Hessian matrix plays a crucial role in the proposed method, and it can be derived by the time differential of $\boldsymbol{J}_\Xi$, which can be written as

$$\dot{\boldsymbol{J}}_\Xi(\boldsymbol{\Phi}, \dot{\boldsymbol{\Phi}}) = \sum_{i=1}^{l_\Phi} \frac{\partial \boldsymbol{J}_\Xi}{\partial \Phi_i} \dot{\Phi}_i. \tag{54}$$

If $\dot{\bar{\boldsymbol{\Phi}}}_i$ is denoted as the vector with zero everywhere except on the $i$th element, which is the unit, i.e.,

$$\dot{\bar{\boldsymbol{\Phi}}}_i = \begin{bmatrix} 0 & \cdots & 0 & 1 & 0 & \cdots & 0 \end{bmatrix}^T, \tag{55}$$

then an algebraic computation yields

$$\boldsymbol{H}_{\Xi_i}(\boldsymbol{\Phi}) = \frac{\partial \boldsymbol{J}_\Xi}{\partial \Phi_i} = \dot{\boldsymbol{J}}_\Xi(\boldsymbol{\Phi}, \dot{\bar{\boldsymbol{\Phi}}}_i). \tag{56}$$

$\dot{\boldsymbol{J}}_\Xi$ can be represented as

$$\dot{\boldsymbol{J}}_\Xi = \begin{bmatrix} \dot{\boldsymbol{J}}^T_{\xi_1} & \dot{\boldsymbol{J}}^T_{\xi_2} & \cdots & \dot{\boldsymbol{J}}^T_{\xi_m} \end{bmatrix}^T. \tag{57}$$

If $\boldsymbol{H}_{\Xi_i}$ is also written as

$$\boldsymbol{H}_{\Xi_i} = \begin{bmatrix} \boldsymbol{H}^T_{\xi_1} & \boldsymbol{H}^T_{\xi_2} & \cdots & \boldsymbol{H}^T_{\xi_m} \end{bmatrix}^T, \tag{58}$$

then we get

$$\boldsymbol{H}_{\xi_k} = \dot{\boldsymbol{J}}_{\xi_k}, \quad k = 1, \cdots, m. \tag{59}$$

For simplicity, we drop the subscript $i$. The following explains how to compute $\dot{\boldsymbol{J}}_\xi$. According to (53), it is derived as

$$\dot{\boldsymbol{J}}_\xi = -\frac{\mathrm{d}}{\mathrm{d}t}\left(\mathrm{dexp}^{-1}(\boldsymbol{\xi})\right)\boldsymbol{J}_\Phi - \mathrm{dexp}^{-1}(\boldsymbol{\xi})\dot{\boldsymbol{J}}_\Phi, \tag{60}$$

where $\mathrm{d}\left(\mathrm{dexp}^{-1}(\boldsymbol{\xi})\right)/\mathrm{d}t$ is given by (80) in Appendix C. Based on (51), $\dot{\boldsymbol{J}}_\Phi$ are written as

$$\dot{\boldsymbol{J}}_\Phi = \begin{cases} \begin{bmatrix} \dot{\boldsymbol{J}}_{\boldsymbol{S}} & \dot{\boldsymbol{J}}_\Gamma & \dot{\boldsymbol{J}}_{\delta \boldsymbol{q}} \end{bmatrix} & \text{for KI,} \\ \dot{\boldsymbol{J}}_{\delta \boldsymbol{q}} & \text{for IK,} \end{cases}$$

where

$$\begin{aligned} \dot{\boldsymbol{J}}_{\boldsymbol{S}} &= \begin{bmatrix} \dot{\boldsymbol{J}}_{S_1} & \dot{\boldsymbol{J}}_{S_2} & \cdots & \dot{\boldsymbol{J}}_{S_n} \end{bmatrix} \in \mathbb{R}^{6\times 6n}, \\ \dot{\boldsymbol{J}}_\Gamma &= \frac{\mathrm{d}}{\mathrm{d}t}\mathrm{dexp}(-\boldsymbol{\Gamma}) \in \mathbb{R}^{6\times 6}, \\ \dot{\boldsymbol{J}}_{\delta \boldsymbol{q}} &= \begin{bmatrix} \dot{\boldsymbol{J}}_{\delta q_1} & \dot{\boldsymbol{J}}_{\delta q_2} & \cdots & \dot{\boldsymbol{J}}_{\delta q_n} \end{bmatrix} \in \mathbb{R}^{6\times n}, \end{aligned}$$

where $\mathrm{d}\left(\mathrm{dexp}^{-1}(-\boldsymbol{\Gamma})\right)/\mathrm{d}t$ is computed from (80) in Appendix C,

$$\begin{aligned} \dot{\boldsymbol{J}}_{S_i} =& \frac{\mathrm{d}}{\mathrm{d}t}\mathrm{Ad}^{-1}_{\boldsymbol{T}_{i\sim n}} \mathrm{dexp}\left(-\boldsymbol{S}_i(\bar{q}_i + \delta q_i)\right)(\bar{q}_i + \delta q_i) \\ &+ \mathrm{Ad}^{-1}_{\boldsymbol{T}_{i\sim n}} \frac{\mathrm{d}}{\mathrm{d}t}\left(\mathrm{dexp}\left(-\boldsymbol{S}_i(\bar{q}_i + \delta q_i)\right)\right)(\bar{q}_i + \delta q_i) \\ &+ \mathrm{Ad}^{-1}_{\boldsymbol{T}_{i\sim n}} \left(\mathrm{dexp}\left(-\boldsymbol{S}_i(\bar{q}_i + \delta q_i)\right)\right)\delta \dot{q}_i, \\ \dot{\boldsymbol{J}}_{\delta q_i} =& \frac{\mathrm{d}}{\mathrm{d}t}\mathrm{Ad}^{-1}_{\boldsymbol{T}_{i\sim n}} \boldsymbol{S}_i + \mathrm{Ad}^{-1}_{\boldsymbol{T}_{i\sim n}} \dot{\boldsymbol{S}}_i, \end{aligned}$$

where $\mathrm{d}\left(\mathrm{dexp}(\cdot)\right)/\mathrm{d}t$ is derived from (79) in Appendix C,

$$\frac{\mathrm{d}}{\mathrm{d}t}\mathrm{Ad}^{-1}_{\boldsymbol{T}_{i\sim n}} = \sum_{k=i+1}^{n+1} \mathrm{Ad}^{-1}_{\boldsymbol{T}_{k\sim n}} \frac{\mathrm{d}}{\mathrm{d}t}\left(\mathrm{Ad}^{-1}_{\boldsymbol{T}_k}\right)\mathrm{Ad}^{-1}_{\boldsymbol{T}_{i\sim k-2}},$$

where based on (83) in Appendix D,

$$\frac{\mathrm{d}}{\mathrm{d}t}\left(\mathrm{Ad}^{-1}_{\boldsymbol{T}_k}\right) = -\mathrm{ad}(\boldsymbol{V}_k)\mathrm{Ad}^{-1}_{\boldsymbol{T}_k}.$$

The above presents a unified calculation method for the Hessian matrix applicable to KI and IK problems. Although the method is implicit, it can be simplified in the IK case to yield an explicit computation procedure, thereby significantly enhancing algorithmic efficiency. Appendix E provides the detailed derivation, constituting another key theoretical contribution.

### C. Summary of DMH Method

The DMH method for KI and IK is illustrated in Algorithm 1. The MATLAB codebase for the DMH method can be viewed and downloaded at https://anonymous.4open.science/r/DMH-06B1. We classify both KI and IK as kinematic problems, and thus, we propose a unified function, denoted as *DMH_Method_KI_IK* in MATLAB, to address both problems consistently. Furthermore, the MH method can be recovered by setting $\sigma = 0$.

## V. Simulations and Experiments

This section evaluates the DMH method for KI and IK via both simulation and experiments. Sections V-A and V-B compare the proposed method with state-of-the-art methods for KI and IK, using serial robots with various configurations. Section V-C shows KI conducted with two robots in real environments. Finally, the analysis and discussion are presented in Section V-D.

**Algorithm 1:** DMH method for KI and IK.

**Data:** Parameters $\boldsymbol{S}^{(0)}$, $\boldsymbol{\Gamma}^{(0)}$, $\delta\boldsymbol{q}^{(0)}$, $\bar{\boldsymbol{q}}_1, \cdots, \bar{\boldsymbol{q}}_m$, $\boldsymbol{T}_{d_1}, \cdots, \boldsymbol{T}_{d_m}$ for KI; parameters $\delta\boldsymbol{q}^{(0)}$, $\boldsymbol{S}$, $\boldsymbol{\Gamma}$, $\bar{\boldsymbol{q}}_1 = 0$, $\boldsymbol{T}_{d_1}$, $m = 1$ for IK; convergence tolerance $\epsilon$; maximum iteration count $\tau_{\max}$; damping factor $\sigma$.
**Result:** $\boldsymbol{S}$, $\boldsymbol{\Gamma}$, $\delta\boldsymbol{q}$ for KI; $\delta\boldsymbol{q}$ for IK.

```
Initialization: τ ← 0 and compute Φ^(τ) using (21) or (24);
repeat
    τ ← τ + 1;
    S^(τ), Γ^(τ), δq^(τ) ← Φ^(τ) using (21) for KI; or
      δq^(τ) ← Φ^(τ) using (24) for IK;
    for i ← 1 to m do
        Compute T_{e_i}^(τ), T_{ξ_i}^(τ), and ξ_i^(τ) using (16), (23), and
          (61), respectively;
        Compute J_{ξ_i}^(τ) using (53);
        for k ← 1 to l_Φ do
            Φ̇_k^(τ) ← Φ̇̄_k computed by (55);
            Ṡ^(τ), Γ̇^(τ), δq̇^(τ) ← Φ̇_k^(τ) using (21) for KI; or
              δq̇^(τ) ← Φ̇_k^(τ) using (24) for IK;
            Compute ξ̇_k^(τ) and J̇_{ξ_k}^(τ) using (44) and (60);
            Compute H_{ξ_k}^(τ) using (59);
        end
        Compute H_{Ξ_i}^(τ) using (58);
    end
    Compute Ξ^(τ) using (22) or (25) and J_Ξ^(τ) using (43);
    Compute ΔΦ_DNR^(τ) and ΔΦ_DTH^(τ) using (38) and (39);
    Compute Ξ_DTH^(τ) and ΔΦ_DMH^(τ) using (40) and (41);
    Compute Φ^(τ+1) using (42) ;
until Ξ^T Ξ < ε or τ > τ_max;
Output: S, Γ, δq ← Φ^(τ+1) using (21) for KI; or
        δq ← Φ^(τ+1) using (24) for IK.
```

TABLE I
SIMULATION #1: IDENTIFIED RESULTS OF SCARA ROBOT (UNIT: m, rad)

| | Nominal [7] | Actual [7] | Identified (FIS) | Identified (DMH) |
|---|---|---|---|---|
| $\boldsymbol{S}_1$ | $\begin{bmatrix}0\\0\\1\\0\\0\\0\end{bmatrix}$ | $\begin{bmatrix}0.01999\\0\\0.9998\\0\\0.01303\\0\end{bmatrix}$ | $\begin{bmatrix}0.02000\\0.00001\\0.9998\\-0.00003\\0.013012\\0\end{bmatrix}$ | $\begin{bmatrix}0.01999\\6.4\times10^{-9}\\0.9998\\2.2\times10^{-8}\\0.01303\\3.3\times10^{-9}\end{bmatrix}$ |
| $\boldsymbol{S}_2$ | $\begin{bmatrix}0\\0\\1\\0\\-0.25\\0\end{bmatrix}$ | $\begin{bmatrix}0\\0.0004\\1\\-0.0003\\-0.25399\\0.000102\end{bmatrix}$ | $\begin{bmatrix}-1.5\times10^{-5}\\0.0003998\\1\\-0.000247\\-0.253888\\0.0001015\end{bmatrix}$ | $\begin{bmatrix}8.4\times10^{-9}\\0.0004\\1\\-0.0003\\-0.25399\\0.000102\end{bmatrix}$ |
| $\boldsymbol{S}_3$ | $\begin{bmatrix}0\\0\\0\\0\\0\\-1\end{bmatrix}$ | $\begin{bmatrix}0\\0\\0\\0.02\\0.0196\\-0.99961\end{bmatrix}$ | $\begin{bmatrix}0\\0\\0\\0.01976\\0.01841\\-0.99964\end{bmatrix}$ | $\begin{bmatrix}2.2\times10^{-10}\\-3.4\times10^{-10}\\-2.0\times10^{-9}\\0.02\\0.0196\\-0.99961\end{bmatrix}$ |
| $\boldsymbol{S}_4$ | $\begin{bmatrix}0\\0\\-1\\0\\0.47\\0\end{bmatrix}$ | $\begin{bmatrix}0.04077\\0.03917\\-0.9984\\-0.02668\\0.504558\\0.018706\end{bmatrix}$ | $\begin{bmatrix}0.040717\\0.0392708\\-0.998399\\-0.026427\\0.5041136\\0.0187509\end{bmatrix}$ | $\begin{bmatrix}0.04077\\0.03917\\-0.9984\\-0.02668\\0.504558\\0.018706\end{bmatrix}$ |

### A. KI Simulation

The state-of-the-art methods [7], [6], and [5] were implemented on a 4-DOF SCARA robot, a 6-DOF Motoman HP20D robot, and a 6-DOF FANUC M-710iC robot, respectively. Accordingly, this subsection compares the proposed method with these state-of-the-art methods on the same robot.

*1) Simulation #1 (SCARA Robot without Noises):* The KI method proposed in [7], denoted by the Finite and Instantaneous Screw (FIS) method, demonstrated the advantages over the local POE method [3] and the global POE method [4] through the SCARA robot. Therefore, we compared only with the FIS method under the same conditions. Specifically, the kinematic parameters of the robot were from [7]. $\boldsymbol{T}_{n+1}$ and $\delta\boldsymbol{q}$ were set to $\mathbf{I}$ and $\mathbf{0}$, respectively. Only $\boldsymbol{S}$ was identified, and $\boldsymbol{\Gamma}$ and $\delta\boldsymbol{q}$ were not involved. The number of measurements was given as six, with values randomly generated. The identified results of both methods are listed in Table I.

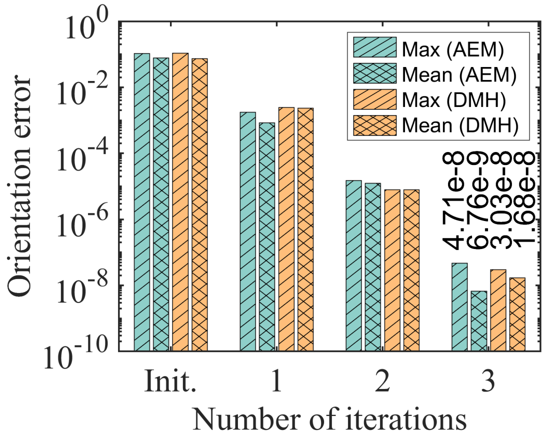
The SCARA robot comprises revolute joints (joints 1, 2, and 4) and a prismatic joint (joint 3). Simulation results indicate that the proposed method is similar to the FIS method, and both can be uniformly applied to the KI of robots, regardless of the type of joints. The error of $\boldsymbol{S}$ identified by the DMH method proposed in this paper was $4.74 \times 10^{-8}$, whereas that of the FIS method was $1.33 \times 10^{-3}$. Moreover, the FIS method required multiple iterations to obtain results, while the DMH method achieved superior accuracy in just one iteration.

*2) Simulation #2 (Motoman HP20D Robot without Noises):* The KI method proposed in [6], denoted by the Adjoint Error Model (AEM) method, has exhibited excellent performance. Therefore, the proposed DMH method was compared with the AEM method under identical conditions using Motoman HP20D Robot, whose kinematic parameters were from [6]. $\boldsymbol{S}$ and $\boldsymbol{\Gamma}$ were identified, and $\delta\boldsymbol{q}$ was not involved. We also randomly generated 20 robot configurations and measured the tool frame poses for identification, while an additional 500 configurations were randomly selected for verification. This process followed the exact procedure described in [6]. Fig. 4 illustrates the mean and maximum errors at the verification samples after each iteration.

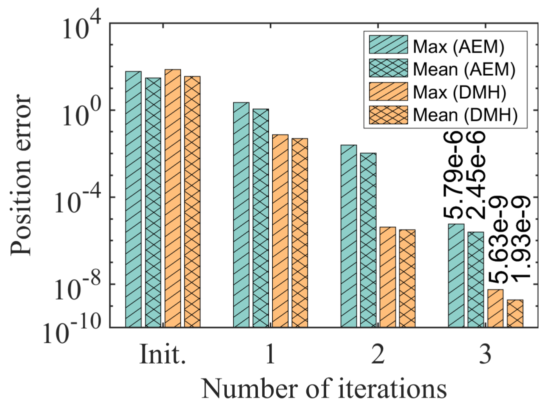
The DMH data were derived from this simulation's computational results, whereas the AEM data were directly sourced from [6]. The initial orientation and position errors computed by the DMH and AEM methods are highly consistent, thereby validating the reliability of our simulation process. For position error (see Fig. 4(b)), the proposed method demonstrates higher identification accuracy ($10^{-9}$ vs. $10^{-6}$). For orientation error (see Fig. 4(a)), both methods present comparable accuracy. The robot's actual parameter $\boldsymbol{T}_{n+1}$ in this simulation, denoted by $\boldsymbol{g}^a_{st_0}$ in [6], does not belong exactly to $SE(3)$ since its rotational component is not an orthonormal matrix

$$\boldsymbol{g}^a_{st_0} = \begin{bmatrix} -0.0230 & -0.9997 & 0.0065 & 1087.27 \\ -0.0320 & -0.0058 & -0.9995 & 13.013 \\ 0.9992 & -0.0232 & -0.0318 & 1399.27 \\ 0 & 0 & 0 & 1 \end{bmatrix} \notin SE(3).$$

This inherently introduces an error into our method, thereby limiting its convergence rate. It is reasonable to predict that, given appropriate robot parameters, the orientation identi-



(a)



(b)

Fig. 4. Simulation #2: Comparison between DMH and AEM in Motoman HP20D robot. (a) Orientation error (in rad). (b) Position error (in mm).

fication accuracy of the DMH method will be improved. Therefore, the DMH method exhibits a better identification accuracy than the AEM method for the same number of iterations. Additionally, the position unit in this simulation is millimeters (mm), and the DMH method does not require additional processing to prevent zigzag convergence.

The DMH, DTH, and DNR methods were also implemented in the above test, and their convergence behaviors are illustrated in Fig. 5. All three methods achieved comparable accuracy after 4, 7, and 7 iterations, respectively. The DMH method does not exhibit a clear advantage in accuracy, primarily due to intrinsic model errors introduced by condition $\boldsymbol{g}_{st_0}^{a} \notin SE(3)$. Further discussion is provided in Simulation #3.

*3) Simulation #3 (FANUC M-710iC Robot without and with Noises):* The KI method proposed in [5], referred to as the reformulated local POE (RLPOE) method, exhibited superior performance compared to both the local POE method [3] and the global POE method [4], as demonstrated using the FANUC M-710iC robot. Therefore, the proposed DHM method was compared with the RLPOE method under identical conditions.

We first verified in the absence of noise. Since the robot's kinematic parameters were initially provided in local POE form in [5], we converted them into global POE parameters, as presented in Table II. $\boldsymbol{S}$ and $\boldsymbol{\Gamma}$ are identified, whereas

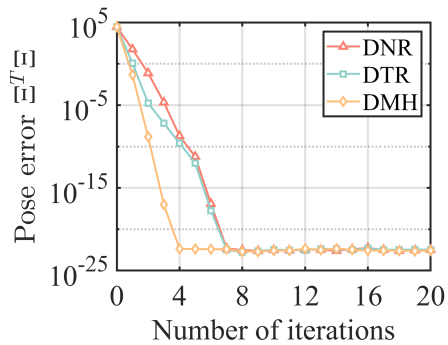



Fig. 5. Simulation #2: Comparison Among DMH, DTH, and DNR in Motoman HP20D robot.

TABLE II

SIMULATION #3: IDENTIFIED RESULTS OF FANUC M710IC ROBOT WITHOUT NOISE (UNIT: m)

| | Nominal [5] | Actual [5] | Identified (DMH) |
|---|---|---|---|
| $\boldsymbol{S}_1$ | $\begin{bmatrix}0\\0\\1\\0\\-0.5\\0\end{bmatrix}$ | $\begin{bmatrix}0\\0\\1\\0\\-0.5\\0\end{bmatrix}$ | $\begin{bmatrix}0.000000\\0.000000\\1.000000\\0.000000\\-0.500000\\0.000000\end{bmatrix}$ |
| $\boldsymbol{S}_2$ | $\begin{bmatrix}0\\1\\0\\-0.565\\0\\0.65\end{bmatrix}$ | $\begin{bmatrix}-0.010527\\0.999919\\0.007087\\-0.557871\\-0.010439\\0.644243\end{bmatrix}$ | $\begin{bmatrix}-0.010527\\0.999919\\0.007087\\-0.557871\\-0.010439\\0.644243\end{bmatrix}$ |
| $\boldsymbol{S}_3$ | $\begin{bmatrix}0\\1\\0\\-1.435\\0\\0.65\end{bmatrix}$ | $\begin{bmatrix}-0.017152\\0.999851\\-0.002000\\-1.422693\\-0.023095\\0.655031\end{bmatrix}$ | $\begin{bmatrix}-0.017152\\0.999851\\-0.002000\\-1.422693\\-0.023095\\0.655031\end{bmatrix}$ |
| $\boldsymbol{S}_4$ | $\begin{bmatrix}1\\0\\0\\0\\1.605\\0\end{bmatrix}$ | $\begin{bmatrix}0.999844\\0.009690\\-0.014794\\-0.015579\\1.602074\\-0.003500\end{bmatrix}$ | $\begin{bmatrix}0.999844\\0.009690\\-0.014794\\-0.015579\\1.602074\\-0.003500\end{bmatrix}$ |
| $\boldsymbol{S}_5$ | $\begin{bmatrix}0\\1\\0\\-1.605\\0\\1.666045\end{bmatrix}$ | $\begin{bmatrix}-0.015943\\0.999846\\0.007277\\-1.567440\\-0.037076\\1.660152\end{bmatrix}$ | $\begin{bmatrix}-0.015943\\0.999846\\0.007277\\-1.567440\\-0.037075\\1.660152\end{bmatrix}$ |
| $\boldsymbol{S}_6$ | $\begin{bmatrix}1\\0\\0\\0\\1.605\\0\end{bmatrix}$ | $\begin{bmatrix}0.999650\\0.018022\\-0.019386\\-0.028841\\1.607866\\0.007549\end{bmatrix}$ | $\begin{bmatrix}0.999650\\0.018022\\-0.019386\\-0.028841\\1.607866\\0.007549\end{bmatrix}$ |
| $\boldsymbol{\Gamma}$ | $\begin{bmatrix}-1.209200\\1.209200\\-1.209200\\0.688952\\-0.252376\\2.629717\end{bmatrix}$ | $\begin{bmatrix}-1.218294\\1.210267\\-1.191684\\0.686072\\-0.231067\\2.623704\end{bmatrix}$ | $\begin{bmatrix}-1.218294\\1.210267\\-1.191684\\0.686072\\-0.231067\\2.623704\end{bmatrix}$ |

$\delta\boldsymbol{q}$ is not considered. 20 poses are randomly generated. The identified results of only one iteration of the DMH methods are given in Table II. The identification errors of $\boldsymbol{S}$ and $\boldsymbol{\Gamma}$ are $1.57\times10^{-8}$ and $3.76\times10^{-9}$, respectively. Achieving this level of accuracy in just one iteration is challenging for currently available methods.

Next, we validated in the presence of noise. In the simulation, 300 poses were randomly generated, with 100 used for parameter identification and the remaining 200 for validation. To verify the robustness, we injected random noise into each simulated measurement with the uniformly distributed range of $[-0.0005, 0.0005]$ rad for orientation and $[-0.0001, 0.0001]$ $m$ for position, respectively. Mean pose errors during the iterative identification process for both DMH and RLPOE methods are presented in Fig. 6. It is easy to see that the RLPOE method requires two iterations to converge, whereas the DMH method converges in just one iteration. Regardless of orientation or position, the proposed DMH method exhibits higher accuracy in identification.

Finally, to evaluate algorithmic effects on KI, DMH and DNR methods were compared under the same cost function in the above scenarios. The comparison results are presented in Fig. 7. In the noise-free scenario, the DMH method enhances the accuracy of KI (see Fig. 7(a)). In the noisy scenario, algorithmic improvements offer limited accuracy gains, but help improve computational efficiency (see Fig. 7(b)).

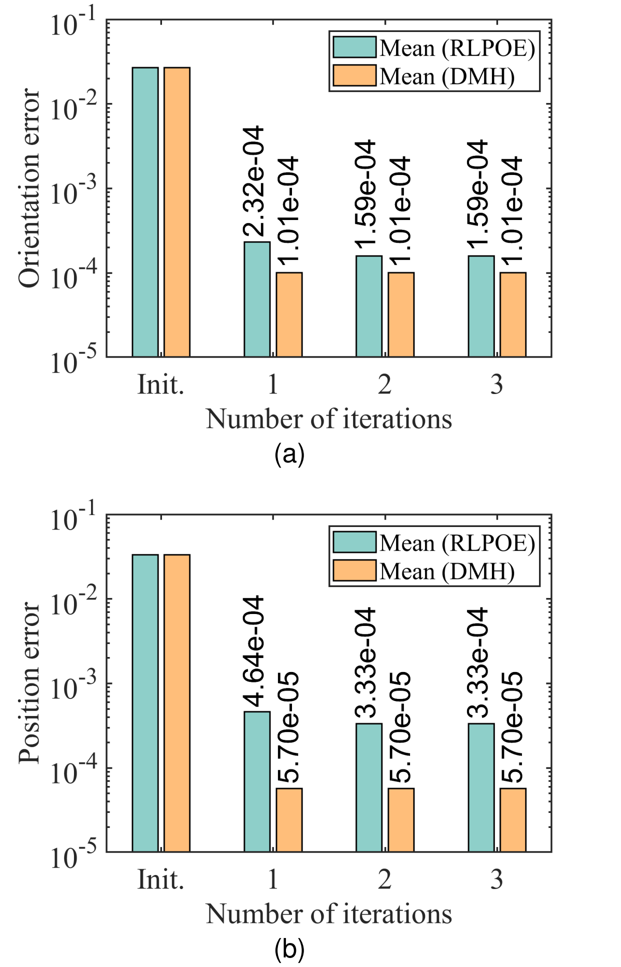



Fig. 6. Simulation #3: Comparison between DMH and RLPOE in FANUC M-710iC robot with noises. (a) Orientation error (in rad). (b) Position error (in m).

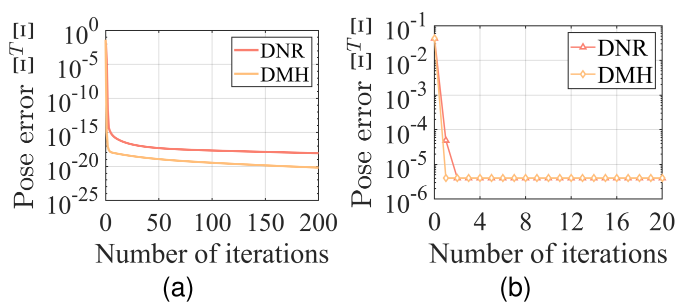



Fig. 7. Simulation #3: Comparison between DMH and DNR in FANUC M-710iC robot. (a) Noise-free scenario. (b) Noisy scenario.

*B. IK Simulation*

This subsection presents a comparative analysis of the proposed method against the state-of-the-art and popular methods for IK with respect to the convergence rate, robustness, and singularity, utilizing various serial robots. All tests were done on a desktop computer (Core i9-14900K 3.2GHz, 128GB RAM) using MATLAB 2023a and C++ code compiled with the MSVC 2019 compiler.

*1) Simulation #4 (Convergence Rate):* The DNR and DQuIK [28] methods are the most common and efficient IK methods, respectively. The update strategy for the DNR method is written as

$$\boldsymbol{\Phi}^{(\tau+1)} = \boldsymbol{\Phi}^{(\tau)} + \Delta\boldsymbol{\Phi}_{\mathrm{DNR}}.$$

The DQuIK method proposed in [28] is based on the $\mathrm{Denavit\text{-}Hartenberg}$ convention, while its equivalent method using the POE formula is the DTH method, with an update strategy as

$$\boldsymbol{\Phi}^{(\tau+1)} = \boldsymbol{\Phi}^{(\tau)} + \Delta\boldsymbol{\Phi}_{\mathrm{DTH}}.$$

Therefore, we will demonstrate the differences in the convergence rates among the DNR, DTH, and DTM methods. As presented in Section III-B, $\boldsymbol{S}$ and $\boldsymbol{\Gamma}$ are known, and $\bar{\boldsymbol{q}} = \boldsymbol{0}$, while $\delta\boldsymbol{q}$ needs to be obtained computationally.

Two robots are used for testing, including the SCARA and FANUC M-710iC robots (see nominal values in Tables I and II for kinematic parameters). We randomly generated $10^5$ target configurations from a uniform distribution within $[-\pi, \pi]$ (rad,m) for each robot. The target end-effector pose was then computed from the forward kinematics, as outlined in (16). The starting joint configuration was obtained by introducing a uniformly distributed perturbation within the range $[-\frac{\pi}{4}, \frac{\pi}{4}]$ (rad,m) to the target configuration. We ran each set of sample joint configurations through the three methods and monitored the pose errors $\Xi^T\Xi$ at each iteration. The number of iterations and the evaluation time (i.e., the CPU time to convergence) are used as metrics to assess convergence.

Fig. 8 presents the results of this test, illustrating a boxplot of iteration errors and their median at each iteration. It is evident that the convergence rate of the DTH method is faster than that of the DNR method, and the convergence trend aligns

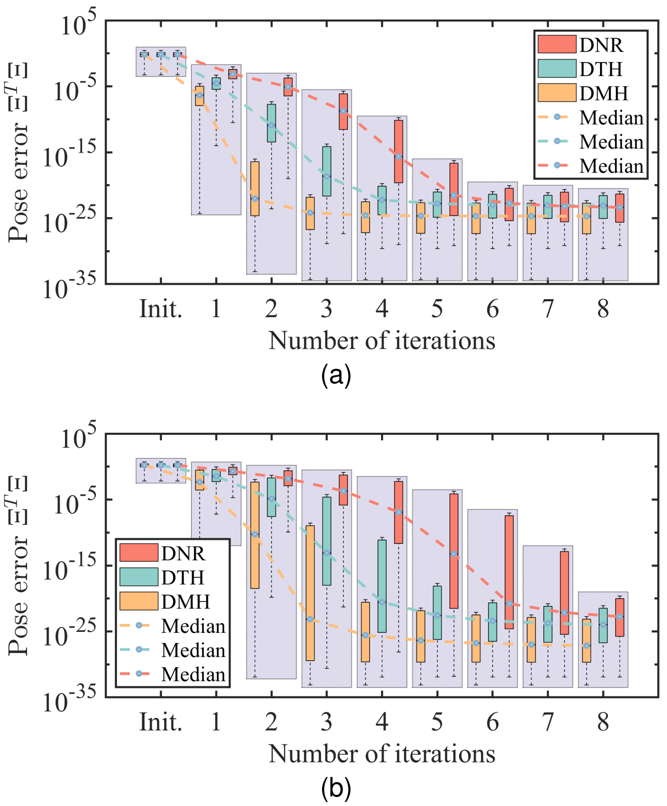



Fig. 8. Simulation #4: Convergence of the DNR, DTH, and DMH methods on various robots. (a) SCARA robot. (b) FANUC M-710iC robot.

with the one reported in [28]. Furthermore, the DMH method is superior to both methods on an iteration-to-iteration basis. The median values of the pose error at each iteration are listed in Table III. For the M-710iC robot, the DNR method converges to a pose error of $10^{-22.7}$ after 8 iterations, while the DTH and DMH methods achieve the same accuracy after 5 and 3 iterations, respectively. Similar conclusions can be drawn for the SCARA robot, as evidenced by the data presented in bold in Table III. Therefore, it is confirmed that the DNR, DTH, and DMH methods exhibit quadratic, cubic, and quintic convergence, respectively.

As demonstrated in [28], a comparative evaluation of DTH and DNR methods reveals that the former achieves significantly superior computational performance. The DMH method may converge faster than the DTH method on an iteration-to-iteration basis; however, since each iteration requires more computations, convergence performance is better indicated by the evaluation time. This test was conducted using a C++ program. Fig. 9 compares the evaluation time for solving the IK of both robots using the DMH and DTM methods. Table IV details the median evaluation time for solving IK using DTH and DMH methods on both robots. Relative to the DTH method, the DMH method exhibits convergence performance gains of $6.57\%$ and $15.79\%$, respectively. Compared to the DTH method, the DMH method requires only one additional forward kinematics and the left inverse of the Jacobian matrix. Therefore, the reduced iteration count of the DMH method relative to the DTH method contributes to its enhanced convergence rate.

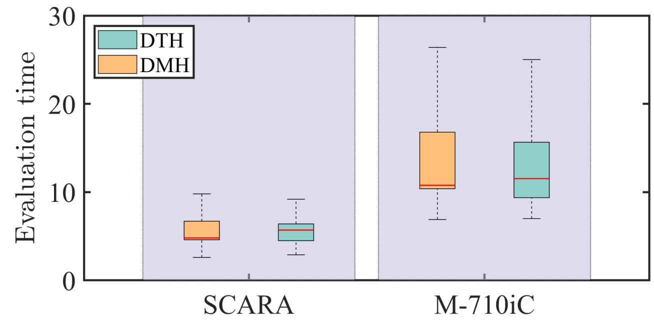



Fig. 9. Simulation #4: Evaluation time of the DTH and DMH methods on FANUC M-710iC robot and SCARA robot (in $\mu$s).

*2) Simulation #5 (Robustness):* Robustness, how often the methods fail, is another crucial metric for evaluating IK methods. To test the metric, joint samples were generated as before but over a range of uniformly distributed perturbations ($[-0.01, 0.01]$ rad to $[-1, 1]$ rad). Methods were run with the convergence tolerance $\epsilon = 10^{-12}$, maximum iteration count $\tau_{\max} = 30$, and damping factor $\sigma = 10^{-6}$. The failure rate is defined as the percentage of cases in which the required accuracy is not achieved. The UR5, FANUC M-710iC, KUKA LBR IIWA 14R820 (see nominal values in Tables II and V for kinematic parameters) robots were used for simulation, with the results for each robot averaged over $10^5$ samples. Each sample was carried out by the DNR, DTH, and proposed DMH methods. These results are plotted in Fig. 10, where Figs. 10(a), 10(b), and 10(c) exhibit the mean number of iterations required to converge for the three robots, while Figs. 10(d), 10(e), and 10(f) present the percentage of samples which failed to converge for each robot.

The results for the mean iterations indicate that the DTH method is superior to the DNR method in all tests, which is consistent with the results in [28]. Furthermore, the DMH method requires the fewest iterations across all tests. The results for the failure rate show that the DMH method is also better than the other two methods for robustness. In addition, as the initial joint error increases, the number of iterations and the failure rate tend to rise. Therefore, a suitable initial value is very necessary for numerical methods. Pretrained lookup tables or machine learning models are proposed to provide approximate solutions, which can then be used as initial inputs for these numerical methods, enabling faster and more reliable convergence to a more accurate result [38].

*3) Simulation #6 (Singularity):* Singularities can affect IK in two ways. One is that the ill-conditioned point is at the initial or intermediate point, and another is at the target point. The former may cause the methods to take a large iterate step length and potentially diverge, or iterate into the wrong solution domain; the latter renders higher-order information useless. To quantify the reliability of singular configurations, a representative singular configuration is generated by setting all joints of the UR5 robot to zero. The configuration is perturbed using random values to obtain a range of condition numbers. A total of $10^5$ samples are generated in this manner. These configurations are subsequently perturbed with values uniformly distributed in the range $[-0.1, 0.1]$ rad to obtain a

TABLE III
SIMULATION #4: MEDIAN OF ERRORS AT EACH ITERATION FOR THE DNR, DTH, AND DMH METHODS

| Iterations | SCARA | | | M-710iC | | |
|---|---|---|---|---|---|---|
| | DNR | DTH | DMH | DNR | DTH | DMH |
| 0 | $10^{-0.2}$ | $10^{-0.2}$ | $10^{-0.2}$ | $10^{0.2}$ | $10^{0.2}$ | $10^{0.2}$ |
| 1 | $10^{-3.1}$ | $10^{-4.5}$ | $10^{-6.4}$ | $10^{-0.7}$ | $10^{-1.5}$ | $10^{-2.3}$ |
| 2 | $10^{-5.1}$ | $10^{-10.9}$ | $10^{-22.0}$ | $10^{-1.8}$ | $10^{-4.9}$ | $10^{-10.3}$ |
| 3 | $10^{-8.7}$ | $10^{-18.6}$ | $\mathbf{10^{-24.2}}$ | $10^{-3.7}$ | $10^{-13.1}$ | $\mathbf{10^{-23.1}}$ |
| 4 | $10^{-15.6}$ | $10^{-22.2}$ | $10^{-24.6}$ | $10^{-6.9}$ | $10^{-20.5}$ | $10^{-25.6}$ |
| 5 | $10^{-21.5}$ | $10^{-22.8}$ | $10^{-24.6}$ | $10^{-13.3}$ | $\mathbf{10^{-22.6}}$ | $10^{-26.4}$ |
| 6 | $10^{-22.8}$ | $10^{-23.0}$ | $10^{-24.7}$ | $10^{-20.8}$ | $10^{-23.4}$ | $10^{-26.8}$ |
| 7 | $10^{-23.2}$ | $10^{-23.1}$ | $10^{-24.7}$ | $10^{-22.1}$ | $10^{-23.8}$ | $10^{-27.0}$ |
| 8 | $\mathbf{10^{-23.4}}$ | $10^{-23.1}$ | $10^{-24.7}$ | $\mathbf{10^{-22.7}}$ | $10^{-24.0}$ | $10^{-27.1}$ |

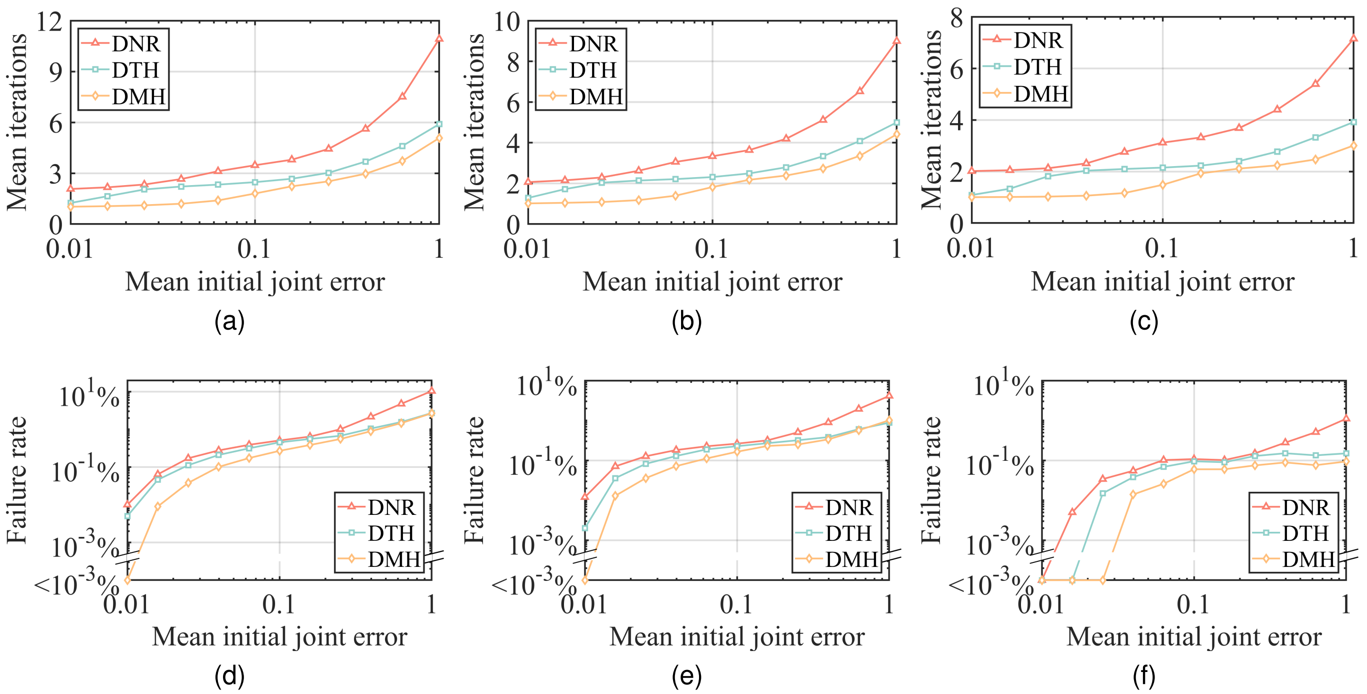



Fig. 10. Simulation #5: Robustness of the DNR, DTH, and DMH methods on various robots. (a) and (d) Mean iterations and failure rate for UR5. (b) and (e) Mean iterations and failure rate for FANUC M-710iC. (c) and (f) Mean iterations and failure rate for KUKA LBR IIWA 14R820.

TABLE IV
SIMULATION #4: MEDIAN EVALUATION TIME FOR SOLVING IK USING DTH AND DMH METHODS ON THE THREE ROBOTS (UNIT: $\mu$s).

| Robots | Methods | | Improvement |
|---|---|---|---|
| | DTH | DMH | |
| M-710iC | 11.53 | 10.77 | **6.57%** ↑ |
| SCARA | 5.70 | 4.80 | **15.79%** ↑ |

second set of configurations. The methods are tested twice: once using the singular poses as the initial guess and the perturbed as the target; again using the singular configurations as the target, and the perturbed configurations as the initial guess.

Figs. 11(a) and 11(b) plots the reliability for both situations over varying condition numbers of $\boldsymbol{J}(\boldsymbol{q})$, with a requested accuracy of $\epsilon = 10^{-8}$. The damped methods clearly outperform the undamped method, and introducing damped significantly

TABLE V
NOMINAL PARAMETERS OF UR5 AND KUKA LBR IIWA 14R820 ROBOTS (UNIT: m)

| | UR5 | KUKA |
|---|---|---|
| $\boldsymbol{S}_1$ | $[0\ \ 0\ \ 1\ \ 0\ \ 0\ \ 0]^T$ | $[0\ \ 0\ \ 1\ \ 0\ \ 0\ \ 0]^T$ |
| $\boldsymbol{S}_2$ | $[0\ \ 1\ \ 0\ \ -0.089\ \ 0\ \ 0]^T$ | $[0\ \ -1\ \ 0\ \ 0.395\ \ 0\ \ 0]^T$ |
| $\boldsymbol{S}_3$ | $[0\ \ 1\ \ 0\ \ -0.089\ \ 0\ \ 0.425]^T$ | $[0\ \ 0\ \ 1\ \ 0\ \ 0\ \ 0]^T$ |
| $\boldsymbol{S}_4$ | $[0\ \ 1\ \ 0\ \ -0.089\ \ 0\ \ 0.817]^T$ | $[0\ \ 1\ \ 0\ \ -0.815\ \ 0\ \ 0]^T$ |
| $\boldsymbol{S}_5$ | $[0\ \ 0\ \ -1\ \ -0.109\ \ 0.817\ \ 0]^T$ | $[0\ \ 0\ \ 1\ \ 0\ \ 0\ \ 0]^T$ |
| $\boldsymbol{S}_6$ | $[0\ \ 1\ \ 0\ \ 0.006\ \ 0\ \ 0.817]^T$ | $[0\ \ -1\ \ 0\ \ 1.215\ \ 0\ \ 0]^T$ |
| $\boldsymbol{S}_7$ | N/A | $[0\ \ 0\ \ 1\ \ 0\ \ 0\ \ 0]^T$ |
| $\boldsymbol{\Gamma}$ | $\begin{bmatrix} 0 \\ 2.221441 \\ 2.221441 \\ 0.218811985 \\ -0.814958840 \\ 0.999958840 \end{bmatrix}$ | $\begin{bmatrix} 0 \\ 0 \\ 0 \\ 0 \\ 0 \\ 1.341 \end{bmatrix}$ |

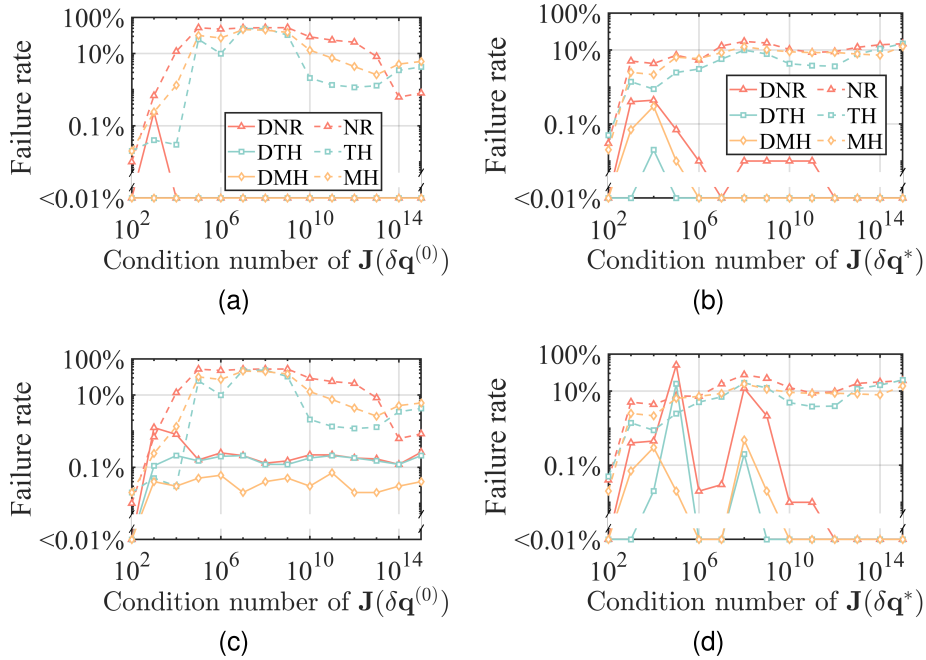



Fig. 11. Simulation #6: Reliability of the NR, TH, MH, DNR, DTH, and DMH methods on the UR5 robot. (a) and (b) Near-singular points are at the initial and target configurations ($\epsilon = 10^{-8}$), respectively. (c) and (d) Near-singular points are at the initial and target configurations ($\epsilon = 10^{-10}$), respectively.

enhances the reliability. Furthermore, the DMH and DTH methods exhibit better performance than the DNR method. The required accuracy is increased to $\epsilon = 10^{-10}$, and the corresponding results are presented in Figs. 11(c) and 11(d). When the singularity is near the initial configuration, the DMH method shows a lower failure rate compared to the DTH and DNR methods, thanks to the proposed improvement strategy.

While damped methods generally improve reliability, appropriate selection of the damped coefficient is essential. The adaptive damped methods (adaptively select the parameter $\sigma$) generally avoid the unstable solution region and improve the performance near singularities. Light and heavy damping provide small and large trust regions, respectively. Damping is a trade-off between convergence speed and robustness. Lighter damping leads to faster convergence but poorer robustness, while heavier damping results in slower convergence but improved robustness. The investigation into the proper selection of $\sigma$ is beyond the scope of the current work. However, the proper tuning of the damping is discussed in [24], [25], and [39].

### C. KI Experiment

This subsection compares the DMH method against the DTH and DNR for KI using various serial robots in real-world experiments.

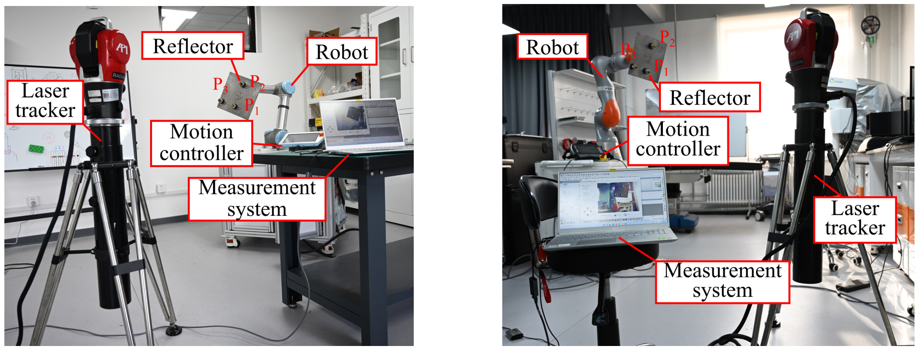
*1) Experimental Setup:* The KI experiments were conducted on the robotic systems available in our laboratory: the UR5 and KUKA LBR IIWA 14R820. The validation system is shown in Fig. 12. A laser tracker (API Radian Pro 20), whose distance measurement accuracy is 10 $\mu$m + 5 $\mu$m/m, was used to measure the pose of the end-effector. A plate with three positioning holes was mounted to the end-effector. The reflectors were attached to the holes to build the tool frame for each measure, with the positions of the reflectors indicated by $\boldsymbol{P}_1$, $\boldsymbol{P}_2$, and $\boldsymbol{P}_3$. In each experiment, 100 samples were randomly generated within the robot workspace, and the motion controller controlled the robot accordingly. Of these, 50 were used to identify the robot parameters, and the rest were used for verification. The samples for KI are illustrated in Fig. 13. The DNR, DTH, and DMH methods were used to identify the actual kinematic parameters.

*2) Experiment #1 (UR5 Robot):* In this experiment, the midpoint between points $\boldsymbol{P}_1$ and $\boldsymbol{P}_2$ was regarded as the origin of the frame $\{e\}$. The $y$-axis was aligned with the line from $\boldsymbol{P}_1$ to $\boldsymbol{P}_2$. Therefore, the $x$-, $y$-, and $z$-axes of the frame $\{e\}$ can be written as

$$\boldsymbol{y} = \frac{\overline{\boldsymbol{P}_1\boldsymbol{P}_2}}{\left\|\overline{\boldsymbol{P}_1\boldsymbol{P}_2}\right\|}, \quad \boldsymbol{z} = \overline{\boldsymbol{P}_1\boldsymbol{P}_2} \times \overline{\boldsymbol{P}_1\boldsymbol{P}_3}, \quad \boldsymbol{x} = \boldsymbol{y} \times \boldsymbol{z}.$$

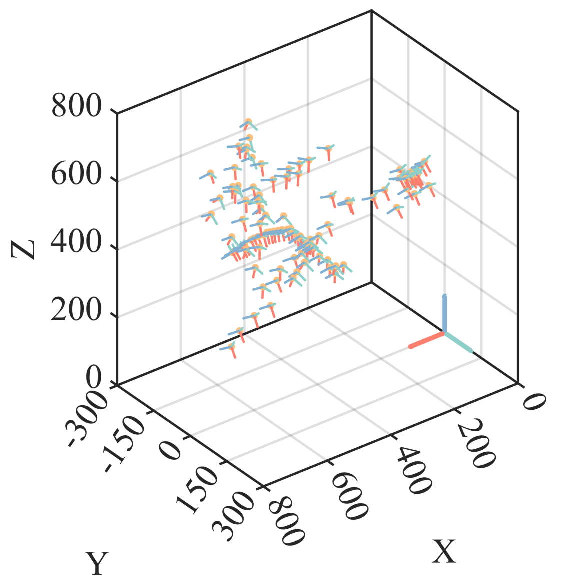
The UR5 robot has six actuated joints whose nominal joint twists at the initial pose are given in Table V.

The result of the experiment is shown in Fig. 14. The initial error is relatively large, as it involves the robot's inherent accuracy error and the estimation error associated with the transformation between the robot base frame and the laser tracker frame. After KI, both orientation and position errors of test points for all three methods are notably reduced. The mean orientation errors for DMH, DTH, and DNR methods are $2.592 \times 10^{-2}$ rad, $2.614 \times 10^{-2}$ rad, and $2.628 \times 10^{-2}$ rad, respectively. The mean position errors are $4.478 \times 10^{-1}$ mm, $4.480 \times 10^{-1}$ mm, and $4.598 \times 10^{-1}$ mm. As a result, the proposed DMH method shows a comparative advantage over DTH and DNR methods, owing to its fifth-order convergence property.

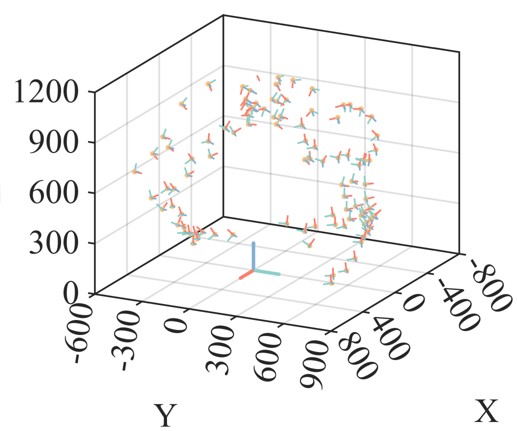



(a) (b)

Fig. 12. Devices for experimental validation. (a) UR5 robot for KI. (b) KUKA LBR IIWA 14R820 for KI.

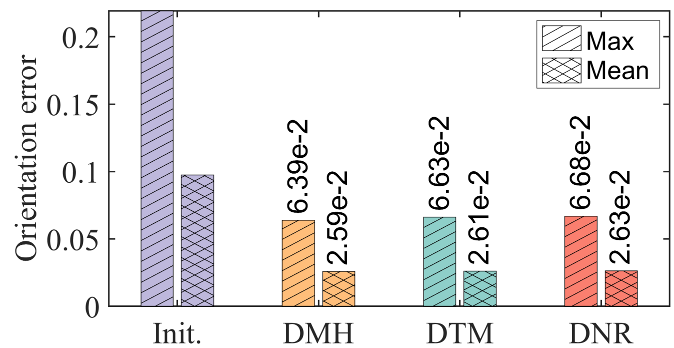



(a)

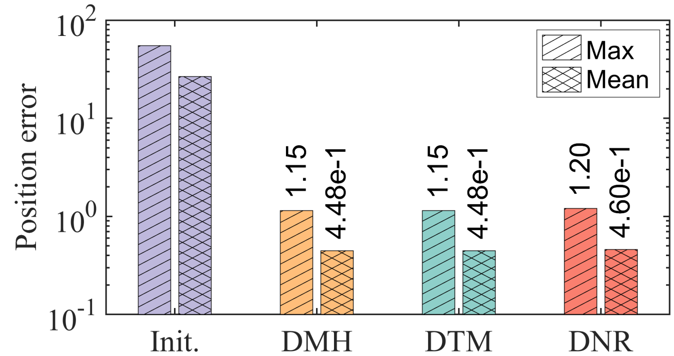



(b)

Fig. 13. Sampled robot poses for the KI. (a) UR5 robot (in mm). (b) KUKA LBR IIWA 14R820 robot (in mm).




(a)




(b)

Fig. 14. Experiment #1: Comparison among DNR, DTH, and DMH for KI in UR5 robot. (a) Orientation error (in rad). (b) Position error (in mm).

*3) Experiment #2 (KUKA LBR IIWA 14R820 Robot):* The midpoint between points $\boldsymbol{P}_1$ and $\boldsymbol{P}_2$ was still considered as the origin of the frame $\{e\}$ in the experiment. The $x$-axis was collinear with the line from $\boldsymbol{P}_1$ to $\boldsymbol{P}_2$. Hence, the $x$-, $y$-, and $z$-axes of the frame $\{e\}$ can be represented as

$$\boldsymbol{x} = \frac{\overline{\boldsymbol{P}_1\boldsymbol{P}_2}}{\left\|\overline{\boldsymbol{P}_1\boldsymbol{P}_2}\right\|}, \quad \boldsymbol{z} = \overline{\boldsymbol{P}_1\boldsymbol{P}_2} \times \overline{\boldsymbol{P}_1\boldsymbol{P}_3}, \quad \boldsymbol{y} = \boldsymbol{z} \times \boldsymbol{x}.$$

The nominal kinematic parameters of the 7-DOF KUKA robot are detailed in Table V.

The results of Experiment #2 are similar to those of Experiment #1. The DMH method provides more satisfactory identification results than the DTH and DNR methods. The

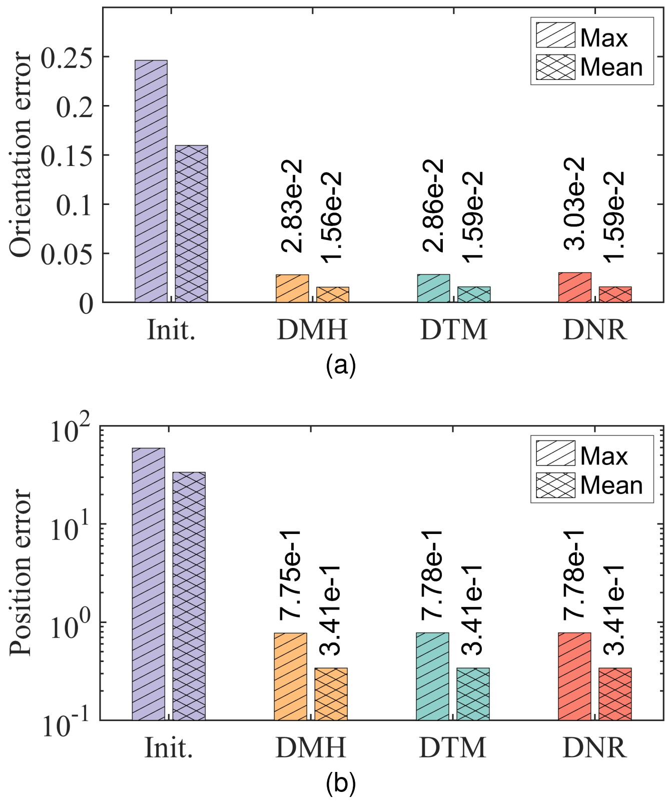



Fig. 15. Experiment #2: Comparison among DNR, DTH, and DMH for KI in KUKA LBR IIWA 14R820 robot. (a) Orientation error (in rad). (b) Position error (in mm).

TABLE VI
DMH AND OTHER METHODS COMPARISON.

| Methods | Simulations | | | | | | Experiments | |
|---|---|---|---|---|---|---|---|---|
| | KI | | | IK | | | KI | |
| | #1 | #2 | #3 | #4 | #5 | #6 | #1 | #2 |
| DMH | ✔ | ✔ | ✔ | ✔ | ✔ | ✔ | ✔ | ✔ |
| FIS | ✤ | ○ | ○ | ○ | ○ | ○ | ○ | ○ |
| AEM | ○ | ✤ | ○ | ○ | ○ | ○ | ○ | ○ |
| RLPOE | ○ | ○ | ✤ | ○ | ○ | ○ | ○ | ○ |
| DNR | ○ | ○ | ○ | ✤ | ✤ | ✤ | ✤ | ✤ |
| DTH | ○ | ○ | ○ | ✤ | ✤ | ✤ | ✤ | ✤ |

Markers ✔, ✤, and ○ mean better performance, acceptable performance, and no comparison, respectively.

DMH method significantly improves the robot's accuracy, with a 9-fold increase in orientation accuracy and a 99-fold increase in positional accuracy (see Figs. 15(a) and 15(b)). The mean orientation and position errors for the DMH method are $1.590 \times 10^{-2}$ rad and $3.410 \times 10^{-1}$ mm after KI, respectively.

### D. Analysis and Discussion

We summarize and analyze the advantages of the proposed DMH method over other methods in solving KI and IK problems using five robots with different configurations. The comparison among DMH and other methods for all tests is detailed in Table VI. The DMH method achieves dual benefits: enhanced identification accuracy for KI and improved convergence/robustness for IK.

First, we summarize the results of KI. Simulation #1 shows that the DMH method successfully performs the KI of the SCARA robot in just one iteration, with accuracy surpassing that of the FIS method with multiple iterations ($4.74 \times 10^{-8}$ vs. $1.33 \times 10^{-3}$). Simulation #2 demonstrates that the DMH method has similar orientation accuracy to the AEM method in the same number of iterations for the Motoman HP20D robot. Still, the position accuracy is significantly superior to the AEM method ($1.93 \times 10^{-9}$ vs. $2.45 \times 10^{-6}$). Simulation #3 exhibits that the DMH method performs the KI of the FANUC M-710iC robot with noises in just one iteration, with orientation and position accuracy surpassing that of the RLPOE method with three iterations ($1.01 \times 10^{-4}$ vs. $1.59 \times 10^{-4}$ and $5.70 \times 10^{-5}$ vs. $3.33 \times 10^{-4}$). Both experiments #1 and #2 for the UR5 and KUKA LBR IIWA 14R820 robots show that the proposed DMH method achieves superior identification accuracy compared to DTH and DNR methods. After KI, the DMH method improved the position accuracy by 60 times and 99 times in the two experiments, while the directional accuracy was enhanced by 4 times and 9 times, respectively. This discrepancy may arise because the orientation of the robot's tool frame is computed based on the positions of the three reflectors. Due to their proximity to each other, the resulting orientation error is magnified. DMH offers moderate accuracy gains over DTH and DNR only in noise-free conditions, while showing limited improvement under noise.

Next, we analyze the results of IK. For Simulation #4, IK for the SCARA and FANUC M-710iC robots demonstrate that the DMH method achieves fewer iterations, faster convergence rate, and higher convergence accuracy than the DTH method. This is because the former method exhibits fifth-order convergence, while the latter method demonstrates third-order convergence, respectively. In addition, the DMH method has only (41) and (42) more than the DTH method. Hence, the DMH method's reduced iteration count compared to the DTH method directly enhances its convergence performance. For Simulation #5, IK for the UR5, FANUC M-710iC, KUKA LBR IIWA 14R820 robots show that the DMH method requires fewer iterations and exhibits a lower failure rate, indicating superior robustness. For Simulation #6, IK for the UR5 robot indicates that the DMH method exhibits superior reliability compared with the DTH and DNR methods for near singular configurations, particularly under high convergence accuracy requirements.

Finally, we provide a general program for solving robot kinematics problems at https://anonymous.4open.science/r/DMH-06B1, which can be used for both KI and IK.

## VI. CONCLUSION

In this paper, we propose a DMH method for KI and IK of serial robots using the POE formula, achieving quintic convergence. The method enhances the identification accuracy for solving KI problems and improves the convergence rate and robustness for solving IK problems. Moreover, it is shown that the Jacobian and Hessian matrices required for the proposed method can be analytically computed based on the time differential of exponential. Six simulations have been conducted to evaluate our method's accuracy, computational efficiency, success rate, and advantages over the state-of-the-art. Two experiments have also been performed to verify the effectiveness of the proposed method. A publicly available codebase, including the MATLAB and C++ implementations of the proposed methods, is provided, enabling readers to reproduce the results presented in this work.

Future work will be devoted to extending the DMH method to other robotic problems that require numerical computation.

## Appendix A
## Matrix Logarithm Mapping $\log(\cdot)$

Given $\boldsymbol{T} \in SE(3)$, we can obtain $\boldsymbol{\lambda}$ to satisfy $\boldsymbol{T} = \exp(\boldsymbol{\lambda}^\wedge)$ based on a matrix logarithm mapping algorithm, i.e.,

$$\boldsymbol{\lambda} = \log(\boldsymbol{T})^\vee. \tag{61}$$

According to (3), we have $\boldsymbol{R} = \exp(\lceil \boldsymbol{\lambda}_\omega \rceil)$ and $\boldsymbol{p} = \mathrm{dexp}_{\boldsymbol{\lambda}_\omega} \boldsymbol{\lambda}_\nu$. The logarithm mapping algorithm is outlined as follows.

(1) If $\boldsymbol{R} = \mathbf{I}$, then $\boldsymbol{\lambda}_\omega = \mathbf{0}$ and $\boldsymbol{\lambda}_\nu = \boldsymbol{p}$.

(2) If the trace of $\boldsymbol{R}$ equal to $-1$ i.e. $\mathrm{trace}(\boldsymbol{R}) = -1$, then $\boldsymbol{\lambda}_\omega$ is any of the following three vectors that is a feasible solution:

$$\begin{aligned}
\boldsymbol{\lambda}_\omega &= \frac{\pi}{\sqrt{2(1+R_{1,1})}} \begin{bmatrix} 1+R_{1,1} & R_{2,1} & R_{3,1} \end{bmatrix}^T, \\
\boldsymbol{\lambda}_\omega &= \frac{\pi}{\sqrt{2(1+R_{2,2})}} \begin{bmatrix} R_{1,2} & 1+R_{2,2} & R_{3,2} \end{bmatrix}^T, \\
\boldsymbol{\lambda}_\omega &= \frac{\pi}{\sqrt{2(1+R_{3,3})}} \begin{bmatrix} R_{1,3} & R_{2,3} & 1+R_{3,3} \end{bmatrix}^T,
\end{aligned} \tag{62}$$

where $R_{i,j}$ is the $i$th row and $j$th column element of $\boldsymbol{R}$, and

$$\boldsymbol{\lambda}_\nu = \mathrm{dexp}^{-1}_{\boldsymbol{\lambda}_\omega} \boldsymbol{p}. \tag{63}$$

(3) Otherwise, we have

$$\boldsymbol{\lambda}_\omega = \left\lfloor \frac{\theta}{2\sin(\theta)} \left( \boldsymbol{R} - \boldsymbol{R}^T \right) \right\rfloor, \tag{64}$$

where

$$\theta = \cos^{-1}\left( \frac{\mathrm{trace}(\boldsymbol{R}) - 1}{2} \right).$$

Then, $\boldsymbol{\lambda}_\nu$ can be computed by (63).

## Appendix B
## Analysis of Convergence for MH

Without loss of generality, we assume that $\boldsymbol{\Xi}$ and $\boldsymbol{\Phi}$ are vectors in $\mathbb{R}^n$. The notation and conventions are as follows:

- $\boldsymbol{H}(\boldsymbol{\Phi}) \in \mathbb{R}^{n\times n\times n}$ denotes the Hessian tensor of $\boldsymbol{\Xi}$ with respect to $\boldsymbol{\Phi}$.
- $\boldsymbol{D}_3(\boldsymbol{\Phi}) \in \mathbb{R}^{n\times n\times n\times n}$ denotes the third derivative tensor (i.e., a rank-4 tensor) of $\boldsymbol{\Xi}$ with respect to $\boldsymbol{\Phi}$.
- Tensor contractions: Let $\boldsymbol{a}, \boldsymbol{b}, \boldsymbol{c} \in \mathbb{R}^n$. Then, $\boldsymbol{H}[\boldsymbol{a}]$, $\boldsymbol{D}_3[\boldsymbol{a}, \boldsymbol{b}] \in \mathbb{R}^{n\times n}$, and $\boldsymbol{H}[\boldsymbol{a}, \boldsymbol{b}]$, $\boldsymbol{D}_3[\boldsymbol{a}, \boldsymbol{b}, \boldsymbol{c}] \in \mathbb{R}^n$.
- Shorthand notation: $\boldsymbol{\Phi}_\tau = \boldsymbol{\Phi}^{(\tau)}$, $\boldsymbol{\Phi}_{\mathrm{TH}_\tau} = \boldsymbol{\Phi}^{(\tau)}_{\mathrm{TH}}$, $\boldsymbol{\Xi}_\tau = \boldsymbol{\Xi}(\boldsymbol{\Phi}^{(\tau)})$, $\boldsymbol{\Xi}_* = \boldsymbol{\Xi}(\boldsymbol{\Phi}^*)$, $\boldsymbol{J}_\tau = \boldsymbol{J}(\boldsymbol{\Phi}^{(\tau)})$, $\boldsymbol{J}_* = \boldsymbol{J}(\boldsymbol{\Phi}^*)$, $\boldsymbol{H}_\tau = \boldsymbol{H}(\boldsymbol{\Phi}^{(\tau)})$, $\boldsymbol{H}_* = \boldsymbol{H}(\boldsymbol{\Phi}^*)$, $\boldsymbol{D}_{3_\tau} = \boldsymbol{D}_3(\boldsymbol{\Phi}^{(\tau)})$, $\boldsymbol{D}_{3_*} = \boldsymbol{D}_3(\boldsymbol{\Phi}^*)$.

Given $\boldsymbol{\Xi}_* = \mathbf{0}$, we set $\boldsymbol{e}_\tau = \boldsymbol{\Phi}_\tau - \boldsymbol{\Phi}^*$ and $\boldsymbol{d}_\tau = \boldsymbol{\Phi}_{\mathrm{TH}_\tau} - \boldsymbol{\Phi}^*$. The MH method proposed in Section III-C can be rewritten as

$$\boldsymbol{\Phi}_{\mathrm{TH}_\tau} = \boldsymbol{\Phi}_\tau - \boldsymbol{M}_\tau^{-1} \boldsymbol{\Xi}_\tau, \tag{65}$$

$$\boldsymbol{\Phi}_{\tau+1} = \boldsymbol{\Phi}_{\mathrm{TH}_\tau} - \boldsymbol{W}_\tau^{-1} \boldsymbol{\Xi}_{\mathrm{TH}_\tau}, \tag{66}$$

where

$$\begin{aligned}
\boldsymbol{\Xi}_{\mathrm{TH}_\tau} &= \boldsymbol{\Xi}(\boldsymbol{\Phi}_{\mathrm{TH}_\tau}), \\
\boldsymbol{M}_\tau &= \boldsymbol{J}_\tau + \frac{1}{2} \boldsymbol{H}_\tau \left[ \Delta \boldsymbol{\Phi}_{\mathrm{NR}_\tau} \right], \\
\boldsymbol{W}_\tau &= \boldsymbol{J}_\tau + \boldsymbol{H}_\tau \left[ \boldsymbol{d}_\tau - \boldsymbol{e}_\tau \right].
\end{aligned}$$

where $\Delta \boldsymbol{\Phi}_{\mathrm{NR}_\tau}$ can be computed by (29). The convergence of the MH method is analyzed below.

Using Taylor expansion about $\Phi^*$, we have

$$\boldsymbol{\Xi}_\tau = \boldsymbol{J}_* \boldsymbol{e}_\tau + \frac{1}{2} \boldsymbol{H}_* \left[ \boldsymbol{e}_\tau^2 \right] + \frac{1}{6} \boldsymbol{D}_{3_*} \left[ \boldsymbol{e}_\tau^3 \right] + \boldsymbol{O}\left( \|\boldsymbol{e}_\tau\|^4 \right). \tag{67}$$

Furthermore, we have

$$\boldsymbol{J}_\tau = \boldsymbol{J}_* + \boldsymbol{H}_* \left[ \boldsymbol{e}_\tau \right] + \frac{1}{2} \boldsymbol{D}_{3_*} \left[ \boldsymbol{e}_\tau^2 \right] + \boldsymbol{O}\left( \|\boldsymbol{e}_\tau\|^3 \right), \tag{68}$$

$$\boldsymbol{H}_\tau = \boldsymbol{H}_* + \boldsymbol{D}_{3_*} \left[ \boldsymbol{e}_\tau \right] + \boldsymbol{O}\left( \|\boldsymbol{e}_\tau\|^2 \right). \tag{69}$$

Using the identity $(\boldsymbol{I} + \boldsymbol{A})^{-1} = \boldsymbol{I} - \boldsymbol{A} + \boldsymbol{A}^2 - \boldsymbol{A}^3 + \cdots$, the inverse of $\boldsymbol{J}_\tau$ can be written as

$$\begin{aligned}
\boldsymbol{J}_\tau^{-1} =& \Bigg( \boldsymbol{J}_* \Bigg( \boldsymbol{I} + \boldsymbol{J}_*^{-1} \boldsymbol{H}_* \left[ \boldsymbol{e}_\tau \right] + \frac{1}{2} \boldsymbol{J}_*^{-1} \boldsymbol{D}_{3_*} \left[ \boldsymbol{e}_\tau^2 \right] \\
& + \boldsymbol{O}\left( \|\boldsymbol{e}_\tau\|^3 \right) \Bigg) \Bigg)^{-1} \\
=& \boldsymbol{J}_*^{-1} - \boldsymbol{J}_*^{-1} \boldsymbol{H}_* \left[ \boldsymbol{e}_\tau \right] \boldsymbol{J}_*^{-1} - \frac{1}{2} \boldsymbol{J}_*^{-1} \boldsymbol{D}_{3_*} \left[ \boldsymbol{e}_\tau^2 \right] \boldsymbol{J}_*^{-1} \\
& + \boldsymbol{J}_*^{-1} \boldsymbol{H}_* \left[ \boldsymbol{e}_\tau \right] \boldsymbol{J}_*^{-1} \boldsymbol{H}_* \left[ \boldsymbol{e}_\tau \right] \boldsymbol{J}_*^{-1} + \boldsymbol{O}\left( \|\boldsymbol{e}_\tau\|^3 \right).
\end{aligned} \tag{70}$$

Injecting (67) and (70) into (29), we have

$$\begin{aligned}
\Delta \boldsymbol{\Phi}_{\mathrm{NR}_\tau} =& -\boldsymbol{e}_\tau + \frac{1}{2} \boldsymbol{J}_*^{-1} \boldsymbol{H}_* \left[ \boldsymbol{e}_\tau^2 \right] + \frac{1}{3} \boldsymbol{J}_*^{-1} \boldsymbol{D}_{3_*} \left[ \boldsymbol{e}_\tau^3 \right] \\
& - \frac{1}{2} \boldsymbol{J}_*^{-1} \boldsymbol{H}_* \left[ \boldsymbol{e}_\tau \right] \boldsymbol{J}_*^{-1} \boldsymbol{H}_* \left[ \boldsymbol{e}_\tau^2 \right] + \boldsymbol{O}\left( \|\boldsymbol{e}_\tau\|^4 \right).
\end{aligned} \tag{71}$$

Based on (68), (69), and (71) , $\boldsymbol{M}_\tau$ can be expressed as

$$\boldsymbol{M}_\tau = \boldsymbol{J}_* + \frac{1}{2} \boldsymbol{H}_* \left[ \boldsymbol{e}_\tau \right] + \frac{1}{4} \boldsymbol{H}_* \left[ \boldsymbol{J}_*^{-1} \boldsymbol{H}_* \left[ \boldsymbol{e}_\tau^2 \right] \right] + \boldsymbol{O}\left( \|\boldsymbol{e}_\tau\|^3 \right). \tag{72}$$

By analogy with (70), we obtain

$$\begin{aligned}
\boldsymbol{M}_\tau^{-1} =& \boldsymbol{J}_*^{-1} - \frac{1}{2} \boldsymbol{J}_*^{-1} \boldsymbol{H}_* \left[ \boldsymbol{e}_\tau \right] \boldsymbol{J}_*^{-1} \\
& - \frac{1}{4} \boldsymbol{J}_*^{-1} \boldsymbol{H}_* \left[ \boldsymbol{J}_*^{-1} \boldsymbol{H}_* \left[ \boldsymbol{e}_\tau^2 \right] \right] \boldsymbol{J}_*^{-1} \\
& + \frac{1}{4} \boldsymbol{J}_*^{-1} \boldsymbol{H}_* \left[ \boldsymbol{e}_\tau \right] \boldsymbol{J}_*^{-1} \boldsymbol{H}_* \left[ \boldsymbol{e}_\tau \right] \boldsymbol{J}_*^{-1} + \boldsymbol{O}\left( \|\boldsymbol{e}_\tau\|^3 \right).
\end{aligned} \tag{73}$$

According to (67) and (73), $\boldsymbol{d}_\tau$ is rewritten as

$$\begin{aligned}\boldsymbol{d}_\tau &= \boldsymbol{\Phi}_\tau - \boldsymbol{M}_\tau^{-1}\boldsymbol{\Xi}_\tau - \boldsymbol{\Phi}^* \\ &= \frac{1}{4}\boldsymbol{J}_*^{-1}\boldsymbol{H}_*\left[\boldsymbol{J}_*^{-1}\boldsymbol{H}_*\left[\boldsymbol{e}_\tau^2\right]\right]\boldsymbol{e}_\tau \\ &\quad - \frac{1}{6}\boldsymbol{J}_*^{-1}\boldsymbol{D}_{3_*}\left[\boldsymbol{e}_\tau^3\right] + \boldsymbol{O}\left(\|\boldsymbol{e}_\tau\|^4\right).\end{aligned} \tag{74}$$

Note that the $\frac{1}{4}\boldsymbol{J}_*^{-1}\boldsymbol{H}_*\left[\boldsymbol{e}_\tau\right]\boldsymbol{J}_*^{-1}\boldsymbol{H}_*\left[\boldsymbol{e}_\tau^2\right]$ term is eliminated.

From (68), (69), and (74), $\boldsymbol{W}_\tau$ is given by

$$\boldsymbol{W}_\tau = \boldsymbol{J}_* - \frac{1}{2}\boldsymbol{D}_{3_*}\left[\boldsymbol{e}_\tau^2\right] + \boldsymbol{O}\left(\|\boldsymbol{e}_\tau\|^3\right). \tag{75}$$

Analogous to (70), we have

$$\boldsymbol{W}_\tau^{-1} = \boldsymbol{J}_*^{-1} + \frac{1}{2}\boldsymbol{J}_*^{-1}\boldsymbol{D}_{3_*}\left[\boldsymbol{e}_\tau^2\right]\boldsymbol{J}_*^{-1} + \boldsymbol{O}\left(\|\boldsymbol{e}_\tau\|^3\right). \tag{76}$$

Taylor expansion of $\boldsymbol{\Xi}_{\mathrm{TH}_\tau}$ about $\boldsymbol{\Phi}^*$ is

$$\boldsymbol{\Xi}_{\mathrm{TH}_\tau} = \boldsymbol{J}_*(\boldsymbol{d}_\tau + \boldsymbol{O}\left(\|\boldsymbol{d}_\tau\|^2\right). \tag{77}$$

Based on (66), we have

$$\begin{aligned}\boldsymbol{e}_{\tau+1} &= \boldsymbol{\Phi}_{\tau+1} - \boldsymbol{\Phi}^* = \boldsymbol{d}_\tau - \boldsymbol{W}_\tau^{-1}\boldsymbol{\Xi}_{\mathrm{TH}_\tau} \\ &= \boldsymbol{C}_{5th} + \boldsymbol{O}\left(\|\boldsymbol{e}_\tau\|^6\right),\end{aligned} \tag{78}$$

where

$$\begin{aligned}\boldsymbol{C}_{5th} = &-\frac{1}{2}\boldsymbol{J}_*^{-1}\boldsymbol{D}_{3_*}\left[\boldsymbol{e}_\tau^2\right] \\ &\left(\frac{1}{4}\boldsymbol{J}_*^{-1}\boldsymbol{H}_*\left[\boldsymbol{J}_*^{-1}\boldsymbol{H}_*\left[\boldsymbol{e}_\tau^2\right]\right]\boldsymbol{e}_\tau - \frac{1}{6}\boldsymbol{J}_*^{-1}\boldsymbol{D}_{3_*}\left[\boldsymbol{e}_\tau^3\right]\right).\end{aligned}$$

Therefore, the MH method is a fifth-order iterative method.

## Appendix C
## Time Differential of $\mathrm{dexp}(\boldsymbol{\lambda})$ and $\mathrm{dexp}^{-1}(\boldsymbol{\lambda})$

The time differential of $\mathrm{dexp}(\boldsymbol{\lambda})$ and $\mathrm{dexp}^{-1}(\boldsymbol{\lambda})$ can be analytically computed as [31, 40, 41]

$$\frac{\mathrm{d}}{\mathrm{d}t}\mathrm{dexp}(\boldsymbol{\lambda}) = \begin{bmatrix} C_{\boldsymbol{\lambda}_\omega}(\dot{\boldsymbol{\lambda}}_\omega) & \boldsymbol{0} \\ \dot{C}(\boldsymbol{\lambda},\dot{\boldsymbol{\lambda}}) & C_{\boldsymbol{\lambda}_\omega}(\dot{\boldsymbol{\lambda}}_\omega)\end{bmatrix}, \tag{79}$$

$$\frac{\mathrm{d}}{\mathrm{d}t}\mathrm{dexp}^{-1}(\boldsymbol{\lambda}) = \begin{bmatrix} D_{\boldsymbol{\lambda}_\omega}(\dot{\boldsymbol{\lambda}}_\omega) & \boldsymbol{0} \\ \dot{D}(\boldsymbol{\lambda},\dot{\boldsymbol{\lambda}}) & D_{\boldsymbol{\lambda}_\omega}(\dot{\boldsymbol{\lambda}}_\omega)\end{bmatrix}, \tag{80}$$

where if $\boldsymbol{\lambda}_\omega = \boldsymbol{0}$, then

$$\begin{aligned}\dot{C}(\boldsymbol{\lambda},\dot{\boldsymbol{\lambda}}) &= \frac{1}{2}\lceil\dot{\boldsymbol{\lambda}}_\nu\rceil + \frac{1}{6}\lceil\dot{\boldsymbol{\lambda}}_\omega,\boldsymbol{\lambda}_\nu\rceil, \\ \dot{D}(\boldsymbol{\lambda},\dot{\boldsymbol{\lambda}}) &= -\frac{1}{2}\lceil\dot{\boldsymbol{\lambda}}_\nu\rceil + \frac{1}{12}\lceil\dot{\boldsymbol{\lambda}}_\omega,\boldsymbol{\lambda}_\nu\rceil,\end{aligned}$$

otherwise,

$$\begin{aligned}\dot{C}(\boldsymbol{\lambda},\dot{\boldsymbol{\lambda}}) =&\frac{\beta}{2}\left(\lceil\dot{\boldsymbol{\lambda}}_\nu\rceil - \boldsymbol{\tau}(\boldsymbol{\lambda})\lceil\boldsymbol{\lambda}_\omega\rceil\right) \\ &+ \Lambda_1\left(\lceil\dot{\boldsymbol{\lambda}}_\nu,\boldsymbol{\lambda}_\omega\rceil + \lceil\dot{\boldsymbol{\lambda}}_\omega,\boldsymbol{\lambda}_\nu\rceil + \tau(\boldsymbol{\lambda})\lceil\boldsymbol{\lambda}_\omega\rceil\right) \\ &+ \Lambda_2\left(\boldsymbol{\delta}_1(\boldsymbol{\lambda},\dot{\boldsymbol{\lambda}}) + \boldsymbol{\tau}(\boldsymbol{\lambda})\lceil\boldsymbol{\lambda}_\omega\rceil^2\right) + \Lambda_3\boldsymbol{\delta}_2(\boldsymbol{\lambda},\dot{\boldsymbol{\lambda}}), \\ \dot{D}(\boldsymbol{\lambda},\dot{\boldsymbol{\lambda}}) =& -\frac{1}{2}\lceil\dot{\boldsymbol{\lambda}}_\nu\rceil + \frac{2(1-\gamma/\beta)}{\|\boldsymbol{\lambda}_\omega\|^4}\boldsymbol{\tau}(\boldsymbol{\lambda})\lceil\boldsymbol{\lambda}_\omega\rceil^2 \\ &+ \Lambda_4(\lceil\dot{\boldsymbol{\lambda}}_\nu,\boldsymbol{\lambda}_\omega\rceil + \lceil\dot{\boldsymbol{\lambda}}_\omega,\boldsymbol{\lambda}_\nu\rceil) + \Lambda_5\boldsymbol{\delta}_3(\boldsymbol{\lambda},\dot{\boldsymbol{\lambda}}),\end{aligned}$$

where

$$\begin{aligned}\tau(\boldsymbol{\lambda}) =&\boldsymbol{\lambda}_\omega^T\boldsymbol{\lambda}_v\frac{\boldsymbol{\lambda}_\omega^T\dot{\boldsymbol{\lambda}}_\omega}{\|\boldsymbol{\lambda}_\omega\|^2}, \\ \boldsymbol{\delta}_1(\boldsymbol{\lambda},\dot{\boldsymbol{\lambda}}) =&\boldsymbol{\lambda}_\omega^T\boldsymbol{\lambda}_\nu\lceil\dot{\boldsymbol{\lambda}}_\omega\rceil + \boldsymbol{\lambda}_\omega^T\dot{\boldsymbol{\lambda}}_\omega\lceil\boldsymbol{\lambda}_\nu\rceil \\ &+ \left(\delta_0(\boldsymbol{\lambda},\dot{\boldsymbol{\lambda}}) - 4\tau(\boldsymbol{\lambda})\right)\lceil\boldsymbol{\lambda}_\omega\rceil, \\ \boldsymbol{\delta}_2(\boldsymbol{\lambda},\dot{\boldsymbol{\lambda}}) =&\boldsymbol{\lambda}_\omega^T\boldsymbol{\lambda}_\nu\lceil\boldsymbol{\lambda}_\omega,\dot{\boldsymbol{\lambda}}_\omega\rceil + \boldsymbol{\lambda}_\omega^T\dot{\boldsymbol{\lambda}}_\omega\lceil\boldsymbol{\lambda}_\omega,\boldsymbol{\lambda}_\nu\rceil \\ &+ \left(\delta_0(\boldsymbol{\lambda},\dot{\boldsymbol{\lambda}}) - 5\tau(\boldsymbol{\lambda})\right)\lceil\boldsymbol{\lambda}_\omega\rceil^2, \\ \boldsymbol{\delta}_3(\boldsymbol{\lambda},\dot{\boldsymbol{\lambda}}) =&\boldsymbol{\lambda}_\omega^T\boldsymbol{\lambda}_\nu\lceil\boldsymbol{\lambda}_\omega,\dot{\boldsymbol{\lambda}}_\omega\rceil + \boldsymbol{\lambda}_\omega^T\dot{\boldsymbol{\lambda}}_\omega\lceil\boldsymbol{\lambda}_\omega,\boldsymbol{\lambda}_\nu\rceil \\ &+ \left(\delta_0(\boldsymbol{\lambda},\dot{\boldsymbol{\lambda}}) - 3\tau(\boldsymbol{\lambda})\right)\lceil\boldsymbol{\lambda}_\omega\rceil^2,\end{aligned}$$

where

$$\delta_0(\boldsymbol{\lambda},\dot{\boldsymbol{\lambda}}) = \dot{\boldsymbol{\lambda}}_\omega^T\boldsymbol{\lambda}_\nu + \boldsymbol{\lambda}_\omega^T\dot{\boldsymbol{\lambda}}_\nu.$$

## Appendix D
## Time Differential of $\mathrm{Ad}_{\boldsymbol{T}}^{-1}$

Based on (12) and (14), we have

$$\mathrm{Ad}_{\boldsymbol{T}}^{-1}\mathrm{Ad}_{\boldsymbol{T}} = \mathrm{Ad}_{\boldsymbol{T}^{-1}\boldsymbol{T}} = \mathbf{I}. \tag{81}$$

The differential of (81) yields

$$\frac{\mathrm{d}}{\mathrm{d}t}\left(\mathrm{Ad}_{\boldsymbol{T}}^{-1}\right)\mathrm{Ad}_{\boldsymbol{T}} + \mathrm{Ad}_{\boldsymbol{T}}^{-1}\frac{\mathrm{d}}{\mathrm{d}t}\left(\mathrm{Ad}_{\boldsymbol{T}}\right) = \boldsymbol{0}. \tag{82}$$

Injecting (15) into (82), we get

$$\frac{\mathrm{d}}{\mathrm{d}t}\left(\mathrm{Ad}_{\boldsymbol{T}}^{-1}\right) = -\mathrm{ad}(\boldsymbol{V})\mathrm{Ad}_{\boldsymbol{T}}^{-1}. \tag{83}$$

## Appendix E
## Explicit Kinematic Hessian Computation for IK Problem

For the IK problem, $\delta\boldsymbol{q}$ is a variable, while $\boldsymbol{S}$ and $\boldsymbol{\Gamma}$ are constants, with $m = 1$ and $\bar{\boldsymbol{q}}_1 = \boldsymbol{0}$. According to (43) and (53), the Jacobian matrix can be simplified to

$$\boldsymbol{J}_\Xi = \boldsymbol{J}_\xi = -\mathrm{dexp}^{-1}(\boldsymbol{\xi})\boldsymbol{J}_{\delta\boldsymbol{q}}. \tag{84}$$

In fact, the computation process inherently requires $\boldsymbol{J}_\Xi^{-1}\boldsymbol{\xi}$ to be determined, which is written as

$$\begin{aligned}\boldsymbol{J}_\Xi^{-1}\boldsymbol{\xi} &= -\boldsymbol{J}_{\delta\boldsymbol{q}}^{-1}\mathrm{dexp}(\boldsymbol{\xi})\boldsymbol{\xi} \\ &= -\boldsymbol{J}_{\delta\boldsymbol{q}}^{-1}\boldsymbol{\xi}.\end{aligned} \tag{85}$$

Therefore, $\boldsymbol{J}_\Xi$ can be replaced by $-\boldsymbol{J}_{\delta\boldsymbol{q}}$, which is a key conclusion that significantly simplifies the following computation. If $\boldsymbol{J}_\Xi$ is represented as

$$\boldsymbol{J}_\Xi = \begin{bmatrix}\boldsymbol{J}_{\Xi_1} & \boldsymbol{J}_{\Xi_2} & \cdots & \boldsymbol{J}_{\Xi_n}\end{bmatrix}, \tag{86}$$

where $\boldsymbol{J}_{\Xi_i}$ is the $i$th column of $\boldsymbol{J}_\Xi$. Based on (52), $\boldsymbol{J}_{\Xi_i}$ is given by

$$\boldsymbol{J}_{\Xi_i} = -\boldsymbol{J}_{\delta q_i} = -\mathrm{Ad}_{\boldsymbol{T}_{i\sim n}}^{-1}\boldsymbol{S}_i. \tag{87}$$

According to (56), $\boldsymbol{H}_{\Xi_j}$ is rewritten as

$$\begin{aligned}\boldsymbol{H}_{\Xi_j} &= \frac{\partial\boldsymbol{J}_\Xi}{\partial\delta q_j} \\ &= \begin{bmatrix}\boldsymbol{H}_{\Xi_{j,1}} & \boldsymbol{H}_{\Xi_{j,2}} & \cdots & \boldsymbol{H}_{\Xi_{j,n}}\end{bmatrix},\end{aligned} \tag{88}$$

where $\boldsymbol{H}_{\Xi_{j,i}}$ is the $i$th column of $\boldsymbol{H}_{\Xi_j}$ and is computed by

$$\boldsymbol{H}_{\Xi_{j,i}} = \frac{\partial \boldsymbol{J}_{\Xi_i}}{\partial \delta q_j} = -\frac{\partial}{\partial \delta q_j}\left(\mathrm{Ad}_{\boldsymbol{T}_{i\sim n}}^{-1}\right)\boldsymbol{S}_i, \tag{89}$$

where $\boldsymbol{T}_{i\sim n}$ is given as (49). It is evident that $\boldsymbol{H}_{\Xi_{j,i}} = \mathbf{0}$ for $j \leq i$. For $j > i$, substituting $t$ in (83) with $\delta q_j$, we have

$$\frac{\partial}{\partial \delta q_j}\left(\mathrm{Ad}_{\boldsymbol{T}_{i\sim n}}^{-1}\right) = -\mathrm{ad}\left(V_{\theta_j}\right)\mathrm{Ad}_{\boldsymbol{T}_{i\sim n}}^{-1}, \tag{90}$$

where

$$\begin{aligned} V_{\theta_j} &= \left(\boldsymbol{T}_{i\sim n}^{-1}\frac{\partial \boldsymbol{T}_{i\sim n}}{\partial \delta q_j}\right)^{\vee} \\ &= \left(\boldsymbol{T}_{n+1}^{-1}\cdots\boldsymbol{T}_{j+1}^{-1}(\boldsymbol{S}_j)^{\wedge}\boldsymbol{T}_{j+1}\cdots\boldsymbol{T}_{n+1}\right)^{\vee} \\ &= \mathrm{Ad}_{\boldsymbol{T}_{j\sim n}}^{-1}\boldsymbol{S}_j. \end{aligned}$$

Injecting (87) and (90) in (89), $\boldsymbol{H}_{\Xi_{j,i}}$ is rewritten as

$$\begin{aligned} \boldsymbol{H}_{\Xi_{j,i}} &= \mathrm{ad}\left(\mathrm{Ad}_{\boldsymbol{T}_{j\sim n}}^{-1}\boldsymbol{S}_j\right)\mathrm{Ad}_{\boldsymbol{T}_{i\sim n}}^{-1}\boldsymbol{S}_i \\ &= \mathrm{ad}\left(\boldsymbol{J}_{\Xi_j}\right)\boldsymbol{J}_{\Xi_i}. \end{aligned} \tag{91}$$

In summary, $\boldsymbol{J}_{\Xi_i}$ is defined as (87) and $\boldsymbol{H}_{\Xi_{j,i}}$ is simplified as

$$\boldsymbol{H}_{\Xi_{j,i}} = \begin{cases} \mathrm{ad}\left(\boldsymbol{J}_{\Xi_j}\right)\boldsymbol{J}_{\Xi_i}, & \text{if } j > i, \\ \mathbf{0}, & \text{otherwise.} \end{cases} \tag{92}$$

The kinematic Hessian matrix $\boldsymbol{H}_{\Xi_j}$ can be computed solely from the Jacobian matrix $\boldsymbol{J}_{\Xi}$, consistent with the conclusion in [28]. In the provided C++ codebase, $\boldsymbol{J}_{\Xi_i}$ and $\boldsymbol{H}_{\Xi_{j,i}}$ are computed according to the above formulas.


## Acknowledgments

Thanks to Shuo Wang for providing language help, to Shanda Wang for helpful discussions, and to all our colleagues for providing all types of help during the preparation of this manuscript.


## References


[1] G. Du, Y. Liang, C. Li, P. X. Liu, and D. Li, "Online robot kinematic calibration using hybrid filter with multiple sensors," *IEEE Transactions on Instrumentation and Measurement*, vol. 69, no. 9, pp. 7092–7107, 2020.

[2] F. C. Park and K. Okamura, "Kinematic calibration and the product of exponentials formula," in *Advances in Robot Kinematics and Computational Geometry*. Springer, 1994, pp. 119–128.

[3] I.-M. Chen, G. Yang, C. T. Tan, and S. H. Yeo, "Local POE model for robot kinematic calibration," *Mechanism and Machine Theory*, vol. 36, no. 11-12, pp. 1215–1239, 2001.

[4] R. He, Y. Zhao, S. Yang, and S. Yang, "Kinematic-parameter identification for serial-robot calibration based on POE formula," *IEEE Transactions on Robotics*, vol. 26, no. 3, pp. 411–423, 2010.

[5] G. Chen, H. Wang, and Z. Lin, "Determination of the identifiable parameters in robot calibration based on the POE formula," *IEEE Transactions on Robotics*, vol. 30, no. 5, pp. 1066–1077, 2014.

[6] C. Li, Y. Wu, H. Löwe, and Z. Li, "POE-based robot kinematic calibration using axis configuration space and the adjoint error model," *IEEE Transactions on Robotics*, vol. 32, no. 5, pp. 1264–1279, 2016.

[7] T. Sun, B. Lian, S. Yang, and Y. Song, "Kinematic calibration of serial and parallel robots based on finite and instantaneous screw theory," *IEEE Transactions on Robotics*, vol. 36, no. 3, pp. 816–834, 2020.

[8] R. Luo, W. Gao, Q. Huang, and Y. Zhang, "An improved minimal error model for the robotic kinematic calibration based on the POE formula," *Robotica*, vol. 40, no. 5, pp. 1607–1626, 2022.

[9] Q. Huang, X. Zhang, H. Pan, K. Lu, and Y. Cheng, "POE-based kinematic calibration for serial robots using left-invariant error representation and decomposed iterative method," *Robotics and Autonomous Systems*, vol. 186, p. 104896, 2025.

[10] D. L. Pieper, *The kinematics of manipulators under computer control*. Stanford University, 1969.

[11] M. Shimizu, H. Kakuya, W.-K. Yoon, K. Kitagaki, and K. Kosuge, "Analytical inverse kinematic computation for 7-DOF redundant manipulators with joint limits and its application to redundancy resolution," *IEEE Transactions on Robotics*, vol. 24, no. 5, pp. 1131–1142, 2008.

[12] Y. Tong, J. Liu, Y. Liu, and Y. Yuan, "Analytical inverse kinematic computation for 7-DOF redundant sliding manipulators," *Mechanism and Machine Theory*, vol. 155, p. 104006, 2021.

[13] N. Kofinas, E. Orfanoudakis, and M. G. Lagoudakis, "Complete analytical forward and inverse kinematics for the NAO humanoid robot," *Journal of Intelligent & Robotic Systems*, vol. 77, pp. 251–264, 2015.

[14] L.-C. Wang and C.-C. Chen, "A combined optimization method for solving the inverse kinematics problems of mechanical manipulators," *IEEE Transactions on Robotics and Automation*, vol. 7, no. 4, pp. 489–499, 1991.

[15] M. Ayyıldız and K. Cetinkaya, "Comparison of four different heuristic optimization algorithms for the inverse kinematics solution of a real 4-DOF serial robot manipulator," *Neural Computing and Applications*, vol. 27, pp. 825–836, 2016.

[16] S. Dereli and R. Köker, "A meta-heuristic proposal for inverse kinematics solution of 7-DOF serial robotic manipulator: quantum behaved particle swarm algorithm," *Artificial Intelligence Review*, vol. 53, pp. 949–964, 2020.

[17] A. D'Souza, S. Vijayakumar, and S. Schaal, "Learning inverse kinematics," in *Proceedings 2001 IEEE/RSJ International Conference on Intelligent Robots and Systems. Expanding the Societal Role of Robotics in the the Next Millennium (Cat. No. 01CH37180)*, vol. 1. IEEE, 2001, pp. 298–303.

[18] B. Bócsi, D. Nguyen-Tuong, L. Csató, B. Schoelkopf, and J. Peters, "Learning inverse kinematics with structured prediction," in *2011 IEEE/RSJ International Conference on Intelligent Robots and Systems*. IEEE, 2011, pp. 698–703.

[19] A. Csiszar, J. Eilers, and A. Verl, "On solving the inverse

kinematics problem using neural networks," in *2017 24th International Conference on Mechatronics and Machine Vision in Practice (M2VIP)*. IEEE, 2017, pp. 1–6.

[20] R. Bensadoun, S. Gur, N. Blau, and L. Wolf, "Neural inverse kinematic," in *International Conference on Machine Learning*. PMLR, 2022, pp. 1787–1797.

[21] P. Beeson and B. Ames, "TRAC-IK: An open-source library for improved solving of generic inverse kinematics," in *2015 IEEE-RAS 15th International Conference on Humanoid Robots (Humanoids)*. IEEE, 2015, pp. 928–935.

[22] A. Aristidou, J. Lasenby, Y. Chrysanthou, and A. Shamir, "Inverse kinematics techniques in computer graphics: A survey," in *Computer graphics forum*, vol. 37, no. 6. Wiley Online Library, 2018, pp. 35–58.

[23] C. W. Wampler, "Manipulator inverse kinematic solutions based on vector formulations and damped least-squares methods," *IEEE Transactions on Systems, Man, and Cybernetics*, vol. 16, no. 1, pp. 93–101, 1986.

[24] S. R. Buss and J.-S. Kim, "Selectively damped least squares for inverse kinematics," *Journal of Graphics Tools*, vol. 10, no. 3, pp. 37–49, 2005.

[25] T. Sugihara, "Solvability-unconcerned inverse kinematics by the Levenberg–Marquardt method," *IEEE Transactions on Robotics*, vol. 27, no. 5, pp. 984–991, 2011.

[26] W. A. Wolovich and H. Elliott, "A computational technique for inverse kinematics," in *The 23rd IEEE Conference on Decision and Control*. IEEE, 1984, pp. 1359–1363.

[27] D. E. Whitney, "Resolved motion rate control of manipulators and human prostheses," *IEEE Transactions on Man-Machine Systems*, vol. 10, no. 2, pp. 47–53, 1969.

[28] S. Lloyd, R. A. Irani, and M. Ahmadi, "Fast and robust inverse kinematics of serial robots using Halley's method," *IEEE Transactions on Robotics*, vol. 38, no. 5, pp. 2768–2780, 2022.

[29] J. Kou, Y. Li, and X. Wang, "A family of fifth-order iterations composed of Newton and third-order methods," *Applied Mathematics and Computation*, vol. 186, no. 2, pp. 1258–1262, 2007.

[30] J. Park and W.-K. Chung, "Geometric integration on euclidean group with application to articulated multibody systems," *IEEE Transactions on Robotics*, vol. 21, no. 5, pp. 850–863, 2005.

[31] A. Müller, "Review of the exponential and Cayley map on SE(3) as relevant for Lie group integration of the generalized Poisson equation and flexible multibody systems," *Proceedings of The Royal Society A*, vol. 477, no. 2253, p. 20210303, 2021.

[32] X. Yang, L. Wu, J. Li, and K. Chen, "A minimal kinematic model for serial robot calibration using POE formula," *Robotics and Computer-Integrated Manufacturing*, vol. 30, no. 3, pp. 326–334, 2014.

[33] L. Wu, X. Yang, K. Chen, and H. Ren, "A minimal POE-based model for robotic kinematic calibration with only position measurements," *IEEE Transactions on Automation Science and Engineering*, vol. 12, no. 2, pp. 758–763, 2014.

[34] L. Wu, R. Crawford, and J. Roberts, "Geometric interpretation of the general poe model for a serial-link robot via conversion into DH parameterization," in *2019 International Conference on Robotics and Automation (ICRA)*. IEEE, 2019, pp. 7360–7366.

[35] J. M. Ortega and W. C. Rheinboldt, *Iterative solution of nonlinear equations in several variables*. Society for Industrial and Applied Mathematics, 2000.

[36] J. Pujol, "The solution of nonlinear inverse problems and the Levenberg-Marquardt method," *Geophysics*, vol. 72, no. 4, pp. W1–W16, 2007.

[37] Y. Nakamura and H. Hanafusa, "Inverse kinematic solutions with singularity robustness for robot manipulator control," *Journal of Dynamic Systems, Measurement, and Control*, vol. 108, no. 3, pp. 163–171, 09 1986.

[38] N. Vahrenkamp, D. Muth, P. Kaiser, and T. Asfour, "IK-Map: An enhanced workspace representation to support inverse kinematics solvers," in *2015 IEEE-RAS 15th International Conference on Humanoid Robots (Humanoids)*. IEEE, 2015, pp. 785–790.

[39] A. A. Maciejewski and C. A. Klein, "Numerical filtering for the operation of robotic manipulators through kinematically singular configurations," *Journal of Robotic Systems*, vol. 5, no. 6, pp. 527–552, 1988.

[40] J. Park and K. Kim, "Tracking on lie group for robot manipulators," in *2014 11th International Conference on Ubiquitous Robots and Ambient Intelligence (URAI)*. IEEE, 2014, pp. 579–584.

[41] V. Sonneville, "A geometric local frame approach for flexible multibody systems," Ph.D. dissertation, ULiège-Université de Liège, 2015.


## Biography Section

**Yuhan Chen** (Member, IEEE) received a Ph.D. degree in mechanical engineering from the Beijing Institute of Technology, Beijing, China, in 2022 and then worked as a postdoctoral fellow at the Southern University of Science and Technology. He has been an assistant professor at the Institute of Robotics and Intelligent Systems at Dalian University of Technology since 2024. His research interests include visual servoing, robotics, and microassembly.

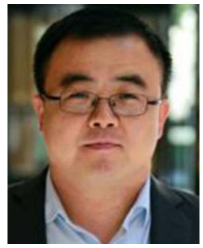
**Yunkai Wang** received the Bachelor's degree in mechanical design, manufacturing, and automation from the School of Mechanical Engineering, Dalian University of Technology, Dalian, China, in 2022. He is currently pursuing a Ph.D. degree at the Institute of Robotics and Intelligent Systems, Dalian University of Technology. His research interests lie in the fields of flexible electronics and robotics.

**Guiyang Xin** (Member, IEEE) received the Ph.D. degree in mechanical engineering from Central South University, Changsha, China, in 2018. He is currently an Associate Professor with Dalian University of Technology, Dalian, China. From 2018 to 2021, he was a Postdoctoral Researcher with the School of Informatics, University of Edinburgh, Edinburgh, U.K. His research interests include legged robotics, impedance control, and optimization-based control.

**Changsheng Dai** (Senior Member, IEEE) received the B.E. and M.E. degrees in mechanical engineering from the Harbin Institute of Technology, China, in 2014 and 2016, respectively, and the Ph.D. degree in mechanical engineering from the University of Toronto in 2020. He was a Post-Doctoral Fellow with the Advanced Micro and Nanosystems Laboratory, University of Toronto, Canada, from 2020 to 2022. He is currently a Professor with the Dalian University of Technology, China. His research interests include robotic cell manipulation and medical robotics.

**Xingjian Liu** (Member, IEEE) received the B.E. degree in electronic information engineering and the Ph.D. degree in materials processing engineering from the Huazhong University of Science and Technology, Wuhan, China, in 2013 and 2018, respectively. He is currently a Professor with the Institute of Robotics and Intelligent Systems (IRIS), Dalian University of Technology. His research interests include computer vision, 3D sensing and metrology, and robotics.

**Yu Sun** (Fellow, IEEE) received the B.S. degree in electrical engineering from the Dalian University of Technology, Dalian, China, in 1996, the M.S. degree in control science and engineering from the Institute of Automation, Chinese Academy of Sciences, Beijing, China, in 1999, and the M.S. and Ph.D. degrees in electrical engineering and mechanical engineering from the University of Minnesota, Minneapolis, in 2001 and 2003, respectively. He is currently a Professor with the University of Toronto. He is also the Tier I Canada Research Chair and the Director of the Robotics Institute. His laboratory specializes in developing innovative technologies and instruments for manipulating and characterizing cells, molecules, and nanomaterials. He was an elected fellow of the American Society of Mechanical Engineers (ASME), American Association for the Advancement of Science (AAAS), National Academy of Inventors (NAI), Canadian Academy of Engineering (CAE), and Royal Society of Canada (RSC), for his work on micro-nano devices and robotic systems. Authorized licensed



**Xinyu Liu** (Member, IEEE) received the B.Eng. and M.Eng. degrees from the Harbin Institute of Technology, Harbin, China, in 2002 and 2004, respectively, and the Ph.D. degree from the University of Toronto, Toronto, ON, Canada, in 2009, all in mechanical engineering. He is a Professor with the Department of Mechanical and Industrial Engineering, University of Toronto, and also with the Institute of Robotics and Intelligent Systems, Dalian University of Technology. Before joining the University of Toronto in September 2017, he was an Associate Professor and the Canada Research Chair in Microfluidics and BioMEMS with the Department of Mechanical Engineering, McGill University, Montreal, QC, Canada. His research interests include microrobotics, nanorobotics, soft robotics, and microfluidics.